MedTrack: AWS Cloud-Enabled Healthcare Management System

Project Description:

In today's fast-evolving healthcare landscape, efficient communication and coordination between doctors and patients are crucial. **MedTrack** is a cloud-based healthcare management system that streamlines patient doctor interactions by providing a centralized platform for booking appointments, managing medical histories, and enabling diagnosis submissions. To address these challenges, the project utilizes Flask for backend development, **AWS EC2** for hosting, and **DynamoDB** for managing data. **MedTrack** allows patients to register, log in, book appointments, and submit diagnosis reports online. The system ensures real-time notifications, enhancing communication between doctors and patients regarding appointments and medical submissions. Additionally, **AWS Identity and Access Management (IAM)** is employed to ensure secure access control to AWS resources, allowing only authorized users to access sensitive data. This cloud-based solution improves accessibility and efficiency in healthcare services for all users.

Scenario 1: Efficient Appointment Booking System for Patients

In the MedTrack system, AWS EC2 provides a reliable infrastructure to manage multiple patients accessing the platform simultaneously. For example, a patient can log in, navigate to the appointment booking page, and easily submit a request for an appointment. Flask handles backend operations, efficiently retrieving and processing user data in real-time. The cloud-based architecture allows the platform to handle a high volume of appointment requests during peak periods, ensuring smooth operation without delays.

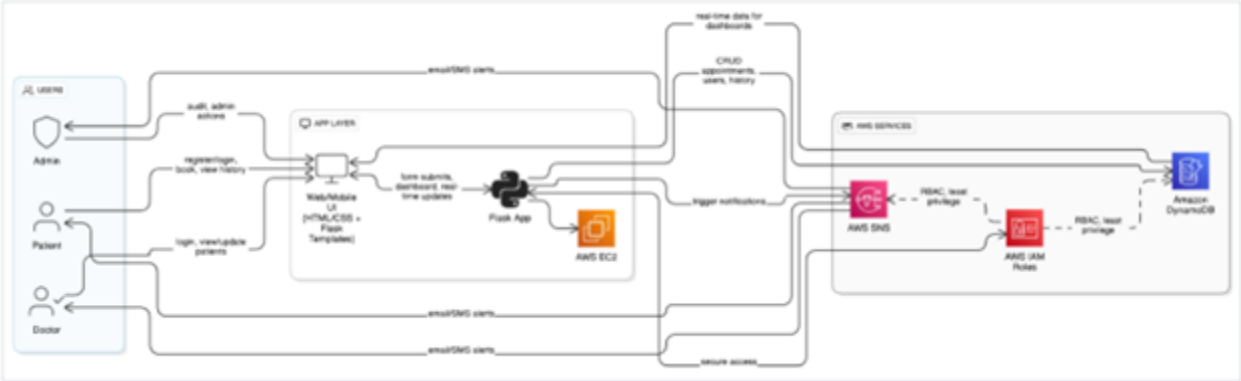
Scenario 2: Secure User Management with IAM

MedTrack utilizes AWS IAM to manage user permissions and ensure secure access to the system. For instance, when a new patient registers, an IAM user is created with specific roles and permissions to access only the features relevant to them. Doctors have their own IAM configurations, allowing them access to patient records and appointment details while maintaining strict security protocols. This setup ensures that sensitive data is accessible only to authorized users.

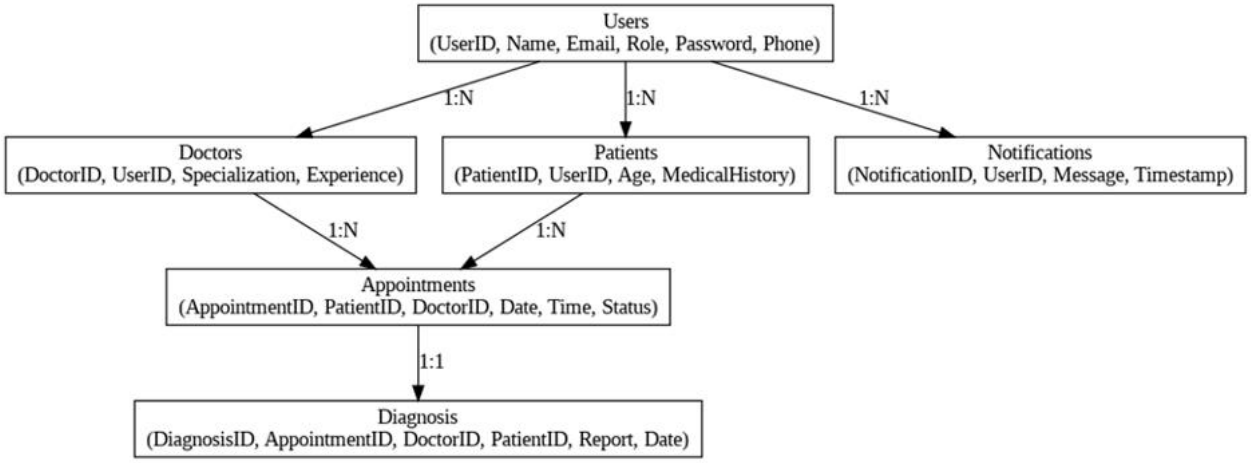
Scenario 3: Easy Access to Medical History and Resources

The MedTrack system provides doctors and patients with easy access to medical histories and relevant resources. For example, a doctor logs in to view a patient's medical history and upcoming appointments. They can quickly access, and update records as needed. Flask manages real-time data fetching from DynamoDB, while EC2 hosting ensures the platform performs seamlessly even when multiple users access it simultaneously, offering a smooth and uninterrupted user experience.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

- AWS Account Setup :
  https://docs.aws.amazon.com/accounts/latest/reference/getting-started.html
- AWS IAM (Identity and Access Management) :
  https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html
- AWS EC2 (Elastic Compute Cloud) :
  https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html
- AWS DynamoDB:

  https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

- Amazon SNS (Simple Notification Service) :
  https://docs.aws.amazon.com/sns/latest/dg/welcome.html
- Git Documentation :
  https://git-scm.com/doc
- VS Code Installation :
  https://code.visualstudio.com/download

Project Work Flow:

Epic 1. Backend Development and Application Setup

Develop the Backend Using Flask.

Integrate AWS Services Using boto3.

Epic 2. AWS Account Setup and Login

Set up an AWS account if not already done.

Log in to the AWS Management Console

Epic 3. DynamoDB Database Creation and Setup

Create a DynamoDB Table.

Configure Attributes for User Data and Book Requests.

Epic 4. SNS Notification Setup

Create SNS topics for book request notifications.

Subscribe users and library staff to SNS email notifications.

Epic 5. IAM Role Setup

Create IAM Role

Attach Policies

Epic 6. EC2 Instance Setup

Launch an EC2 instance to host the Flask application.

Configure security groups for HTTP, and SSH access.

Epic 7. Deployment on EC2

Upload Flask Files

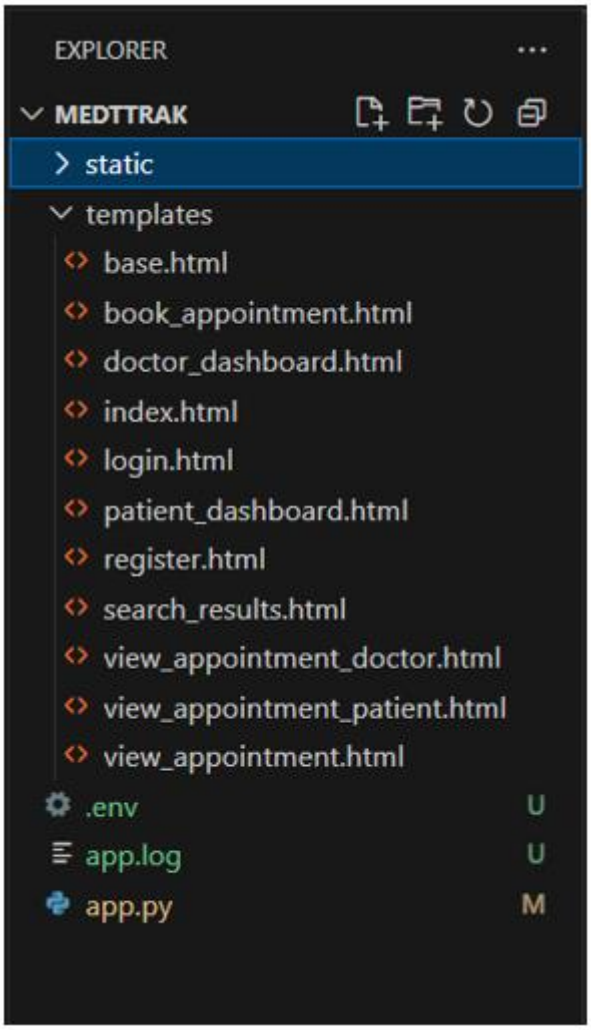
Run the Flask App

Epic 8. Testing and Deployment

Conduct functional testing to verify user registration, login, book requests, and notifications.

Milestone 1: Backend Development and Application Setup

- **Flask Deployment**
  - File Explorer Structure



- **Description of the code:**

  - Flask App Initialization - In the MedTrack project, the Flask app is initialized to establish the backend infrastructure, enabling it to handle multiple user interactions such as patient registration, appointment booking, and submission of medical reports. The Flask framework processes incoming requests, communicates with the DynamoDB database for storing user data, and integrates seamlessly with AWS services. Additionally, the routes and APIs are defined to manage different functionalities like secure login, appointment scheduling, and medical history retrieval. This initialization sets up the foundation for smooth, real-time communication between patients and doctors while ensuring the app is

  — scalable and secure.

```python
from flask import Flask, request, session, redirect, url_for, render_template, flash
import boto3
from werkzeug.security import generate_password_hash, check_password_hash
from datetime import datetime
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
import logging
import os
import uuid
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# ------------------------------------
# Flask App Initialization
# ------------------------------------
app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY', 'temporary_key_for_development')

# ------------------------------------
# App Configuration
# ------------------------------------
AWS_REGION_NAME = os.environ.get('AWS_REGION_NAME', 'ap-south-1')

# Email Configuration
SMTP_SERVER = os.environ.get('SMTP_SERVER', 'smtp.gmail.com')
SMTP_PORT = int(os.environ.get('SMTP_PORT', 587))
SENDER_EMAIL = os.environ.get('SENDER_EMAIL')
SENDER_PASSWORD = os.environ.get('SENDER_PASSWORD')
ENABLE_EMAIL = os.environ.get('ENABLE_EMAIL', 'False').lower() == 'true'

# Table Names from .env
USERS_TABLE_NAME = os.environ.get('USERS_TABLE_NAME', 'UsersTable')
APPOINTMENTS_TABLE_NAME = os.environ.get('APPOINTMENTS_TABLE_NAME', 'AppointmentsTable')
```

```
app = Flask(__name__)
```

Use **boto3** to connect to **DynamoDB** for handling user registration, book requests database operations and also mention region_name where Dynamodb tables are created.

SNS and Dynamodb initialization

In the MedTrack project, AWS SNS sends real-time notifications to patients and doctors about appointments and updates. DynamoDB stores user data, medical records, and appointments securely, offering fast, scalable access. Both services are integrated with Flask to ensure smooth communication and efficient data management.

```
app.py M ×   .env U   app.log U   base.html   view_appointm
app.py > ...
# SNS Configuration
SNS_TOPIC_ARN = os.environ.get('SNS_TOPIC_ARN')
ENABLE_SNS = os.environ.get('ENABLE_SNS', 'False').lower() == 'true'

# ----------------------------------------
# AWS Resources
# ----------------------------------------
dynamodb = boto3.resource('dynamodb', region_name=AWS_REGION_NAME)
sns = boto3.client('sns', region_name=AWS_REGION_NAME)

# DynamoDB Tables
user_table = dynamodb.Table(USERS_TABLE_NAME)
appointment_table = dynamodb.Table(APPOINTMENTS_TABLE_NAME)

# ----------------------------------------
# Logging
# ----------------------------------------
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("app.log"),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)

# ----------------------------------------
# Helper Functions
# ----------------------------------------
def is_logged_in():
    return 'email' in session

def get_user_role(email):
    try:
        response = user_table.get_item(Key={'email': email})
```

SNS Connection

Configure SNS to send notifications when a book request is submitted. Paste your stored ARN link in the sns_topic_arn space, along with the region_name where the SNS topic is created. Also, specify the chosen email service in SMTP_SERVER (e.g., Gmail, Yahoo, etc.) and enter the subscribed email in the SENDER_EMAIL section. Create an 'App password' for the email ID and store it in the SENDER_PASSWORD section.

app.py M × | .env U | app.log U | base.html | view_appointment_doctor.html

app.py > ...

```python
def get_user_role(email):
    except Exception as e:
        logger.error(f"Error fetching role: {e}")
    return None

def send_email(to_email, subject, body):
    if not ENABLE_EMAIL:
        logger.info(f"[Email Skipped] Subject: {subject} to {to_email}")
        return

    try:
        msg = MIMEMultipart()
        msg['From'] = SENDER_EMAIL
        msg['To'] = to_email
        msg['Subject'] = subject
        msg.attach(MIMEText(body, 'plain'))

        server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
        server.starttls()
        server.login(SENDER_EMAIL, SENDER_PASSWORD)
        server.sendmail(SENDER_EMAIL, to_email, msg.as_string())
        server.quit()

        logger.info(f"Email sent to {to_email}")
    except Exception as e:
        logger.error(f"Email sending failed: {e}")

def publish_to_sns(message, subject="Salon Notification"):
    if not ENABLE_SNS:
        logger.info("[SNS Skipped] Message: {}".format(message))
        return

    try:
        response = sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Message=message,
```

```python
            Message=message,
            Subject=subject
        )
        logger.info(f"SNS published: {response['MessageId']}")
    except Exception as e:
        logger.error(f"SNS publish failed: {e}")
```

Routes for Web Pages

● **Register User:**

The **login route** handles user input for authentication, verifying credentials against stored data in **DynamoDB**. On successful login, it increments the **login count** and redirects the user to the appropriate dashboard. This process ensures secure and efficient access to the platform.

```python
# Home Page
@app.route('/')
def index():
    if is_logged_in():
        return redirect(url_for('dashboard'))
    return render_template('index.html')

# Register User (Doctor/Patient)
@app.route('/register', methods=['GET', 'POST'])
def register():
    if is_logged_in():  # Check if already logged in
        return redirect(url_for('dashboard'))
    if request.method == 'POST':
        # Form validation
        required_fields = ['name', 'email', 'password', 'age', 'gender', 'role']
        for field in required_fields:
            if field not in request.form or not request.form[field]:
                flash(f'Please fill in the {field} field', 'danger')
                return render_template('register.html')

        # Check if passwords match
        if 'confirm_password' in request.form and request.form['password'] != request.form['confirm_password']:
            flash('Passwords do not match', 'danger')
            return render_template('register.html')

        name = request.form['name']
        email = request.form['email']
        password = generate_password_hash(request.form['password'])  # Hash password
        age = request.form['age']
        gender = request.form['gender']
        role = request.form['role']  # 'doctor' or 'patient'

        # Check if user already exists
        existing_user = user_table.get_item(Key={'email': email}).get('Item')
        if existing_user:
            flash('Email already registered', 'danger')
            return render_template('register.html')
```

Login Route (GET/POST):

The **login route** handles user authentication by verifying credentials stored in **DynamoDB**. Upon successful login, it increments the **login count** and redirects the user to their dashboard. This ensures secure access to the platform while maintaining user activity logs.

app.py M × | .env U | app.log U | base.html | view_appointment_doctor.html | view_appointment_patient.html

app.py > register

```python
def register():
        # Send welcome email if enabled
        if ENABLE_EMAIL:
            welcome_msg = f"Welcome to HealthCare App, {name}! Your account has been created successfully."
            send_email(email, "Welcome to HealthCare App", welcome_msg)

        # Send admin notification via SNS if configured
        if app.config.get('SNS_TOPIC_ARN'):
            try:
                sns.publish(
                    TopicArn=app.config.get('SNS_TOPIC_ARN'),
                    Message=f'New user registered: {name} ({email}) as {role}',
                    Subject='New User Registration - HealthCare App'
                )
            except Exception as e:
                logger.error(f"Failed to publish to SNS: {e}")

        flash('Registration successful. Please log in.', 'success')
        return redirect(url_for('login'))
    return render_template('register.html')

# Login User (Doctor/Patient)
@app.route('/login', methods=['GET', 'POST'])
def login():
    if is_logged_in():  # If the user is already logged in, redirect to dashboard
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        if not request.form.get('email') or not request.form.get('password') or not request.form.get('role'):
            flash('All fields are required', 'danger')
            return render_template('login.html')

        email = request.form['email']
        password = request.form['password']
        role = request.form['role']  # Get the selected role (doctor or patient)

        # Validate user credentials
```

app.py M × | .env U | app.log U | base.html | view_appointment_doctor.html | view_appointment_patient.html

app.py > register

```python
def login():
        user = user_table.get_item(Key={'email': email}).get('Item')

        if user:
            # Check password and role
            if check_password_hash(user['password'], password):  # Use check_password_hash to verify hashed password
                if user['role'] == role:
                    session['email'] = email
                    session['role'] = role  # Store the role in the session
                    session['name'] = user.get('name', '')

                    # Update login count
                    try:
                        user_table.update_item(
                            Key={'email': email},
                            UpdateExpression='SET login_count = if_not_exists(login_count, :zero) + :inc',
                            ExpressionAttributeValues={':inc': 1, ':zero': 0}
                        )
                    except Exception as e:
                        logger.error(f"Failed to update login count: {e}")

                    flash('Login successful.', 'success')
                    return redirect(url_for('dashboard'))
                else:
                    flash('Invalid role selected.', 'danger')
            else:
                flash('Invalid password.', 'danger')
        else:
            flash('Email not found.', 'danger')

    return render_template('login.html')

# Logout User
@app.route('/logout')
def logout():
    session.pop('email', None)
    session.pop('role', None)
```

Logout Route:

The logout functionality allows users to securely end their session, clearing any session data and redirecting them to the login page. The dashboard provides users with an overview of their activities, such as upcoming appointments for patients or patient records for doctors, with relevant actions based on user roles.

```
app.py M ×   .env U   app.log U   base.html   view_appointment_doctor.html
app.py > register
def logout():
    flash('You have been logged out.', 'success')
    return redirect(url_for('index'))

# Dashboard for both Doctors and Patients
@app.route('/dashboard')
def dashboard():
    if not is_logged_in():
        flash('Please log in to continue.', 'danger')
        return redirect(url_for('login'))

    role = session['role']
    email = session['email']

    if role == 'doctor':
        # Use GSI instead of scan for better performance
        try:
            response = appointment_table.query(
                IndexName='DoctorEmailIndex',
                KeyConditionExpression="doctor_email = :email",
                ExpressionAttributeValues={":email": email}
            )
            appointments = response['Items']
        except Exception as e:
            logger.error(f"Failed to fetch appointments: {e}")
            # Fallback to scan if GSI is not yet created
            try:
                response = appointment_table.scan(
                    FilterExpression="#doctor_email = :email",
                    ExpressionAttributeNames={"#doctor_email": "doctor_email"},
                    ExpressionAttributeValues={":email": email}
                )
                appointments = response['Items']
            except Exception as ex:
                logger.error(f"Fallback scan failed: {ex}")
                appointments = []
```

```python
def dashboard():
        return render_template('doctor_dashboard.html', appointments=appointments)

    elif role == 'patient':
        # Use GSI instead of scan for better performance
        try:
            response = appointment_table.query(
                IndexName='PatientEmailIndex',
                KeyConditionExpression="patient_email = :email",
                ExpressionAttributeValues={":email": email}
            )
            appointments = response['Items']
        except Exception as e:
            logger.error(f"Failed to query appointments: {e}")
            # Fallback to scan if GSI is not yet created
            try:
                response = appointment_table.scan(
                    FilterExpression="#patient_email = :email",
                    ExpressionAttributeNames={"#patient_email": "patient_email"},
                    ExpressionAttributeValues={":email": email}
                )
                appointments = response['Items']
            except Exception as ex:
                logger.error(f"Fallback scan failed: {ex}")
                appointments = []

        # Get list of doctors for booking new appointments
        try:
            doctor_response = user_table.scan(
                FilterExpression="#role = :role",
                ExpressionAttributeNames={"#role": "role"},
                ExpressionAttributeValues={":role": 'doctor'}
            )
            doctors = doctor_response['Items']
        except Exception as e:
            logger.error(f"Failed to fetch doctors: {e}")
            doctors = []
```

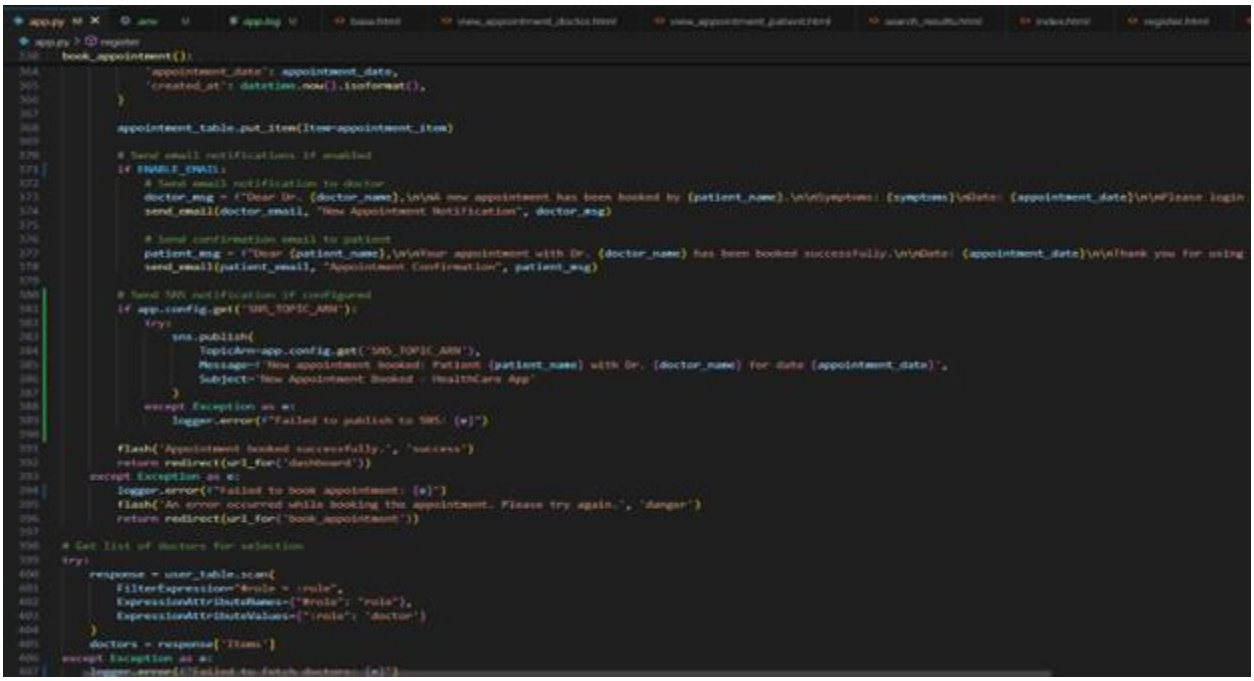
Book Appointment Route:

The book appointment route allows users to select a date, time, and doctor for their appointment. Upon submission, the system stores the appointment details in DynamoDB and sends a confirmation notification via SNS. This ensures smooth scheduling and timely updates for both patients and doctors.

app.py M ✕ | .env U | app.log U | base.html | view_appointment_doctor.html | view_ap

app.py > register

```python
# Book an Appointment (Patient)
@app.route('/book_appointment', methods=['GET', 'POST'])
def book_appointment():
    if not is_logged_in() or session['role'] != 'patient':
        flash('Only patients can book appointments.', 'danger')
        return redirect(url_for('login'))

    if request.method == 'POST':
        # Form validation
        if not request.form.get('doctor_email') or not request.form.get('symptoms'):
            flash('Please fill all required fields', 'danger')
            return redirect(url_for('book_appointment'))

        doctor_email = request.form['doctor_email']
        symptoms = request.form['symptoms']
        appointment_date = request.form.get('appointment_date', datetime.now().isoformat())
        patient_email = session['email']

        # Get patient and doctor information for notifications
        try:
            patient = user_table.get_item(Key={'email': patient_email}).get('Item', {})
            doctor = user_table.get_item(Key={'email': doctor_email}).get('Item', {})

            patient_name = patient.get('name', 'Patient')
            doctor_name = doctor.get('name', 'Doctor')

            # Create a new appointment
            appointment_id = str(uuid.uuid4())
            appointment_item = {
                'appointment_id': appointment_id,
                'doctor_email': doctor_email,
                'doctor_name': doctor_name,
                'patient_email': patient_email,
                'patient_name': patient_name,
                'symptoms': symptoms,
                'status': 'pending',
```



View Appointment Route:

The view appointment route allows users to access a list of their upcoming appointments. It retrieves appointment details from DynamoDB and displays them on the user's dashboard, providing information like date, time, and doctor. This ensures users can easily track their scheduled appointments.

```
app.py M ×   .env U   app.log U   base.html   view_appointment_doctor.html   view_appo
app.py > register
330  book_appointment():
399  try:
400      response = user_table.scan(
401          FilterExpression="#role = :role",
402          ExpressionAttributeNames={"#role": "role"},
403          ExpressionAttributeValues={":role": 'doctor'}
404      )
405      doctors = response['Items']
406  except Exception as e:
407      logger.error(f"Failed to fetch doctors: {e}")
408      doctors = []
409  return render_template('book_appointment.html', doctors=doctors)
410
411  ew Appointment (Doctor)
412  .route('/view_appointment/<appointment_id>', methods=['GET', 'POST'])
413  view_appointment(appointment_id):
414  if not is_logged_in():
415      flash('Please log in to continue.', 'danger')
416      return redirect(url_for('login'))
417
418  # Fetch appointment details
419  try:
420      response = appointment_table.get_item(Key={'appointment_id': appointment_id})
421      appointment = response.get('Item')
422
423      if not appointment:
424          flash('Appointment not found.', 'danger')
425          return redirect(url_for('dashboard'))
426
427      # Security check - verify the logged-in user should access this appointment
428      if session['role'] == 'doctor' and appointment['doctor_email'] != session['email']:
429          flash('You are not authorized to view this appointment.', 'danger')
430          return redirect(url_for('dashboard'))
431      elif session['role'] == 'patient' and appointment['patient_email'] != session['email']:
432          flash('You are not authorized to view this appointment.', 'danger')
433          return redirect(url_for('dashboard'))
434
435      if request.method == 'POST' and session['role'] == 'doctor':
436          diagnosis = request.form['diagnosis']
437          treatment_plan = request.form['treatment_plan']
438          prescription = request.form['prescription']
439
440          # Update appointment with doctor's diagnosis and treatment plan
441          appointment_table.update_item(
442              Key={'appointment_id': appointment_id},
```

Search Appointment Route:

The **search appointment route** enables users to search for specific appointments by date, doctor, or status. It queries **DynamoDB** to fetch relevant appointment details and displays the results in real-time, allowing users to quickly find specific appointments from their history or upcoming schedule.

```
app.py M ×   aws   app.log   base.html   view_appointment_doctor.html   view_appointment_patient.html   search_results.html   index.html   register.html
app.py > register
view_appointment(appointment_id):
            ExpressionAttributeValues={
                ':d': diagnosis,
                ':t': treatment_plan,
                ':p': prescription,
                ':s': 'completed',
                ':u': datetime.now().isoformat()
            },
            ExpressionAttributeNames={
                '#s': 'status'  # Use #s as an alias for the reserved keyword 'status'
            }
        )

        # Send email notification to patient if enabled
        if ENABLE_EMAIL:
            patient_email = appointment['patient_email']
            patient_name = appointment.get('patient_name', 'Patient')
            doctor_name = appointment.get('doctor_name', 'your doctor')

            patient_msg = f"Dear {patient_name},\n\nYour appointment with Dr. {doctor_name} has been completed.\n\nDiagnosis: {diagnosis}\n\nTreatment Plan: {treatment_plan}\
            send_email(patient_email, "Appointment Completed - Diagnosis Available", patient_msg)

        flash('Diagnosis submitted successfully.', 'success')
        return redirect(url_for('dashboard'))

    # Determine which template to render based on user role
    if session['role'] == 'doctor':
        return render_template('view_appointment_doctor.html', appointment=appointment)
    else:  # patient
        return render_template('view_appointment_patient.html', appointment=appointment)
except Exception as e:
    logger.error(f"Error in view_appointment: {e}")
    flash('An error occurred. Please try again.', 'danger')
    return redirect(url_for('dashboard'))

arch functionality for appointments
.route('/search_appointments', methods=['GET', 'POST'])
search_appointments():
if not is_logged_in():
    flash('Please log in to continue.', 'danger')
    return redirect(url_for('login'))

if request.method == 'POST':
    search_term = request.form.get('search_term', '')
```

Profile Route:

The **profile route** allows users to view and update their personal information, such as name, contact details, and medical history. It retrieves user data from **DynamoDB** and displays it on the dashboard. Users can also make changes to their profile, which are then saved securely in the database.

```python
search_appointments():
    try:
        if session['role'] == 'doctor':
            # Doctors can search their patients by name
            response = appointment_table.scan(
                FilterExpression="#doctor_email = :email AND contains(#patient_name, :search)",
                ExpressionAttributeNames={
                    "#doctor_email": "doctor_email",
                    "#patient_name": "patient_name"
                },
                ExpressionAttributeValues={
                    ":email": session['email'],
                    ":search": search_term
                }
            )
        else:  # patient
            # Patients can search their appointments by doctor name or status
            response = appointment_table.scan(
                FilterExpression="#patient_email = :email AND (contains(#doctor_name, :search) OR contains(#status, :search))",
                ExpressionAttributeNames={
                    "#patient_email": "patient_email",
                    "#doctor_name": "doctor_name",
                    "#status": "status"
                },
                ExpressionAttributeValues={
                    ":email": session['email'],
                    ":search": search_term
                }
            )

        appointments = response['Items']
        return render_template('search_results.html', appointments=appointments, search_term=search_term)
    except Exception as e:
        logger.error(f"Search failed: {e}")
        flash('Search failed. Please try again.', 'danger')

return redirect(url_for('dashboard'))

er profile page
.route('/profile', methods=['GET', 'POST'])
profile():
if not is_logged_in():
    flash('Please log in to continue.', 'danger')
    return redirect(url_for('login'))
```

Submit Diagnosis Route:

The submit diagnosis route allows users (patients) to upload and submit their diagnosis reports. Upon submission, the system stores the diagnosis data in DynamoDB and notifies the assigned doctor via SNS. This ensures that medical reports are securely recorded and shared for timely review and treatment.

app.py M ✕ .env U app.log U base.html view_appointment_doctor.ht

app.py > register

```python
profile():
email = session['email']
try:
    user = user_table.get_item(Key={'email': email}).get('Item', {})

    if request.method == 'POST':
        # Update user profile
        name = request.form.get('name')
        age = request.form.get('age')
        gender = request.form.get('gender')

        update_expression = "SET #name = :name, age = :age, gender = :gender"
        expression_values = {
            ':name': name,
            ':age': age,
            ':gender': gender
        }

        # Update specialization only for doctors
        if session['role'] == 'doctor' and 'specialization' in request.form:
            update_expression += ", specialization = :spec"
            expression_values[':spec'] = request.form['specialization']

        user_table.update_item(
            Key={'email': email},
            UpdateExpression=update_expression,
            ExpressionAttributeValues=expression_values,
            ExpressionAttributeNames={'#name': 'name'}
        )

        # Update session name
        session['name'] = name

        flash('Profile updated successfully.', 'success')
        return redirect(url_for('profile'))

    return render_template('profile.html', user=user)
except Exception as e:
    logger.error(f"Profile error: {e}")
    flash('An error occurred. Please try again.', 'danger')
    return redirect(url_for('dashboard'))

bmit Diagnosis (Doctor) - For form submission
.route('/submit_diagnosis/<appointment_id>', methods=['POST'])
submit_diagnosis(appointment_id):
```

Health Routing And Deployment Code :

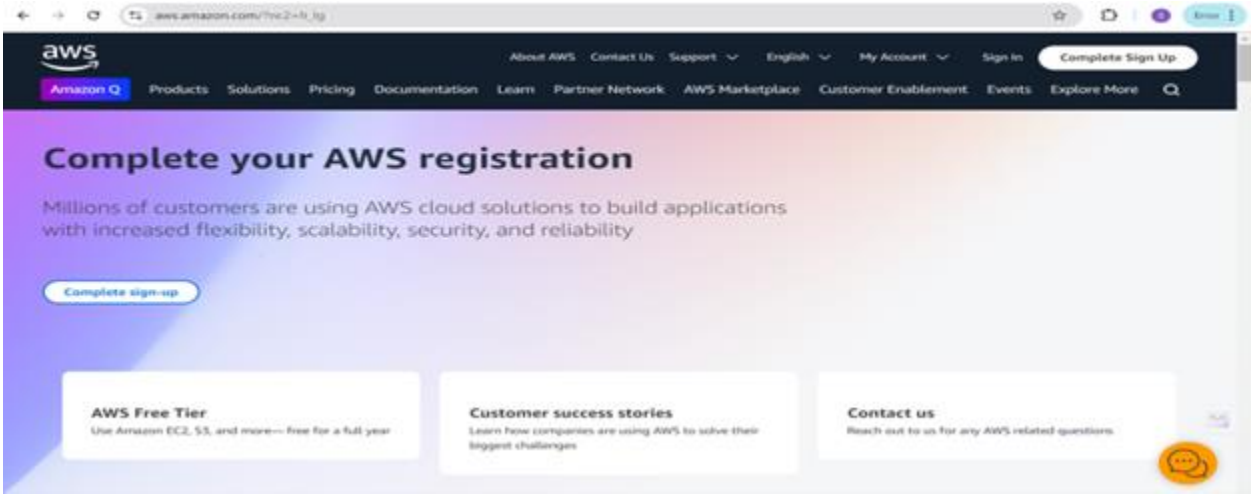
The health routing feature in the MedTrack project checks the system's status by sending a request to a specific endpoint, ensuring the backend services are functioning properly. The name == '__main__' block is used in the Flask app to ensure that the application runs only if the script is executed

directly, not when imported as a module, enabling local development or deployment on a server. This setup ensures that the app runs smoothly and is self-contained during execution.

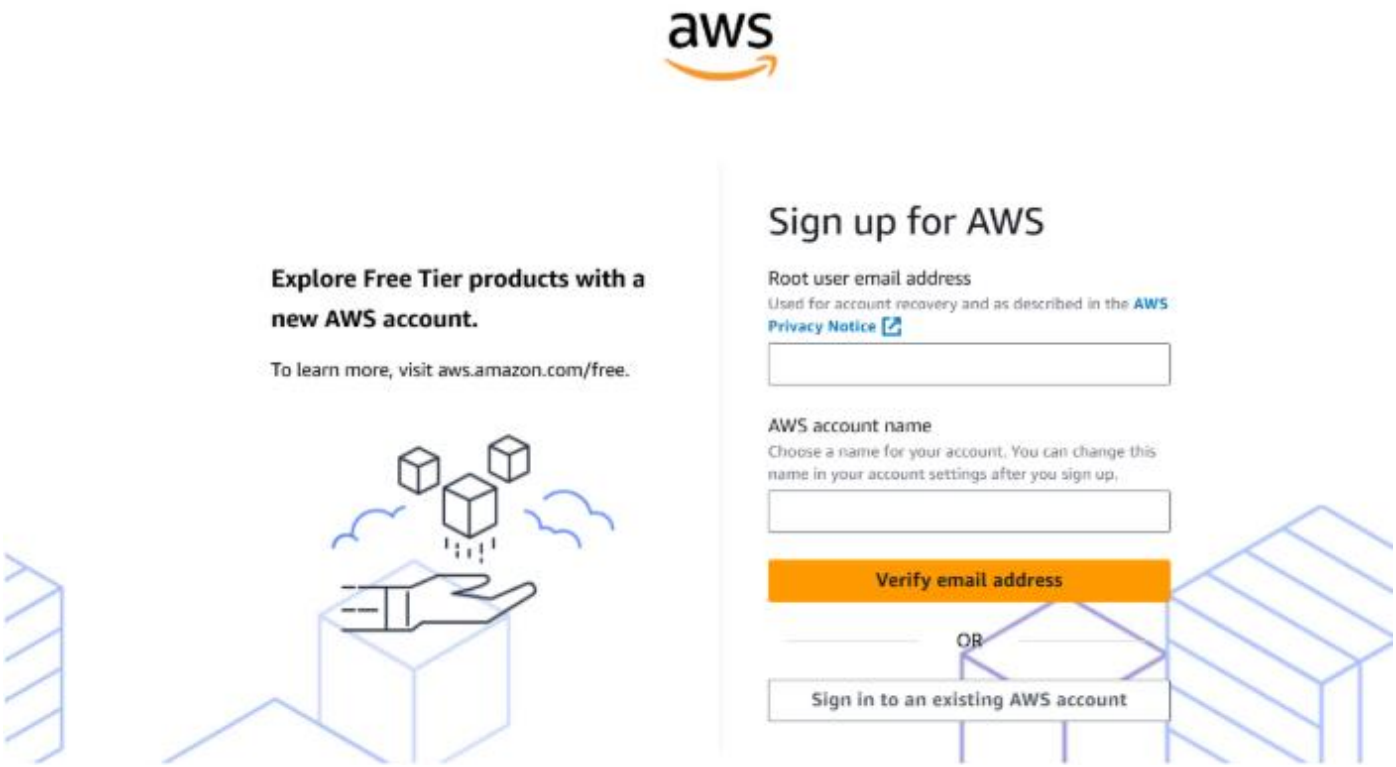
```
app.py M ×   .env   app.log   base.html   view_appointment_doctor.html   view_appointment_patient.html   search_results.html   index.html   register.html
app.py > register
575 submit_diagnosis(appointment_id):
605     )
606
607     # Send email notification to patient if enabled
608     if ENABLE_EMAIL:
609         patient_email = appointment['patient_email']
610         patient_name = appointment.get('patient_name', 'Patient')
611         doctor_name = session.get('name', 'your doctor')
612
613         patient_msg = f"Dear {patient_name},\n\nYour appointment with Dr. {doctor_name} has been completed.\n\nDiagnosis: {diagnosis}\n\nTreatment Plan: {treatment_plan}\n\nP
614         send_email(patient_email, "Appointment Completed - Diagnosis Available", patient_msg)
615
616     flash('Diagnosis submitted successfully.', 'success')
617     return redirect(url_for('dashboard'))
618 except Exception as e:
619     logger.error(f"Submit diagnosis error: {e}")
620     flash('An error occurred while submitting the diagnosis. Please try again.', 'danger')
621     return redirect(url_for('view_appointment', appointment_id=appointment_id))
622
623 alth check endpoint for AWS load balancers
624 .route('/health')
625 health():
626 return {'status': 'healthy'}, 200
627
628 ----------
629 e the Flask app
630 ----------
631 _name__ == '__main__':
632 port = int(os.environ.get('PORT', 5000))
633 debug_mode = os.environ.get('FLASK_ENV') == 'development'
634 app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

Milestone 2: AWS Account Setup and Login

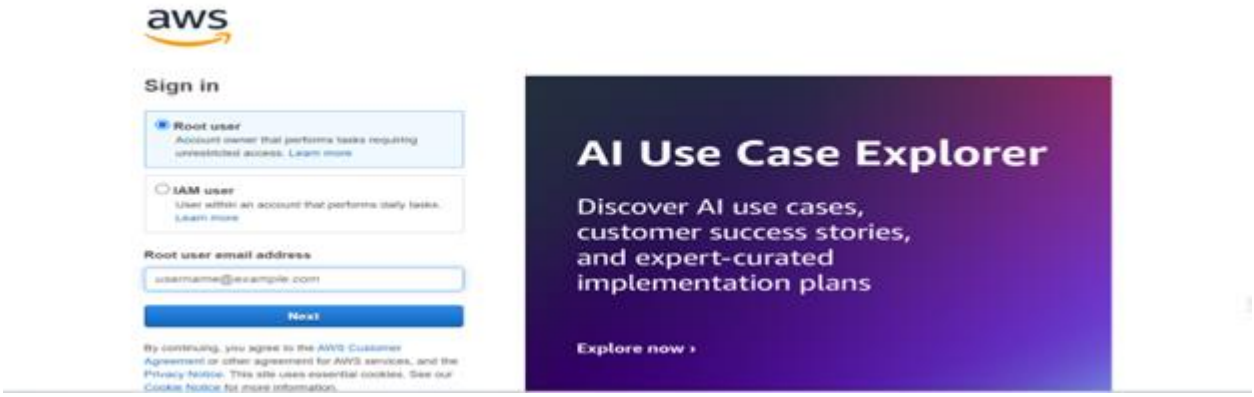
- This is for your understanding only, please refrain from creating an AWS account. A temporary account will be provided via Troven.
- Go to the AWS website (https://aws.amazon.com/).
- Click on the "Create an AWS Account" button.
- Follow the prompts to enter your email address and choose a password.
- Provide the required account information, including your name, address, and phone number.
- Enter your payment information. (Note: While AWS offers a free tier, a credit card or debit card is required for verification.)
- Complete the identity verification process.
- Choose a support plan (the basic plan is free and sufficient for starting).
- Once verified, you can sign in to your new AWS accounts.



.



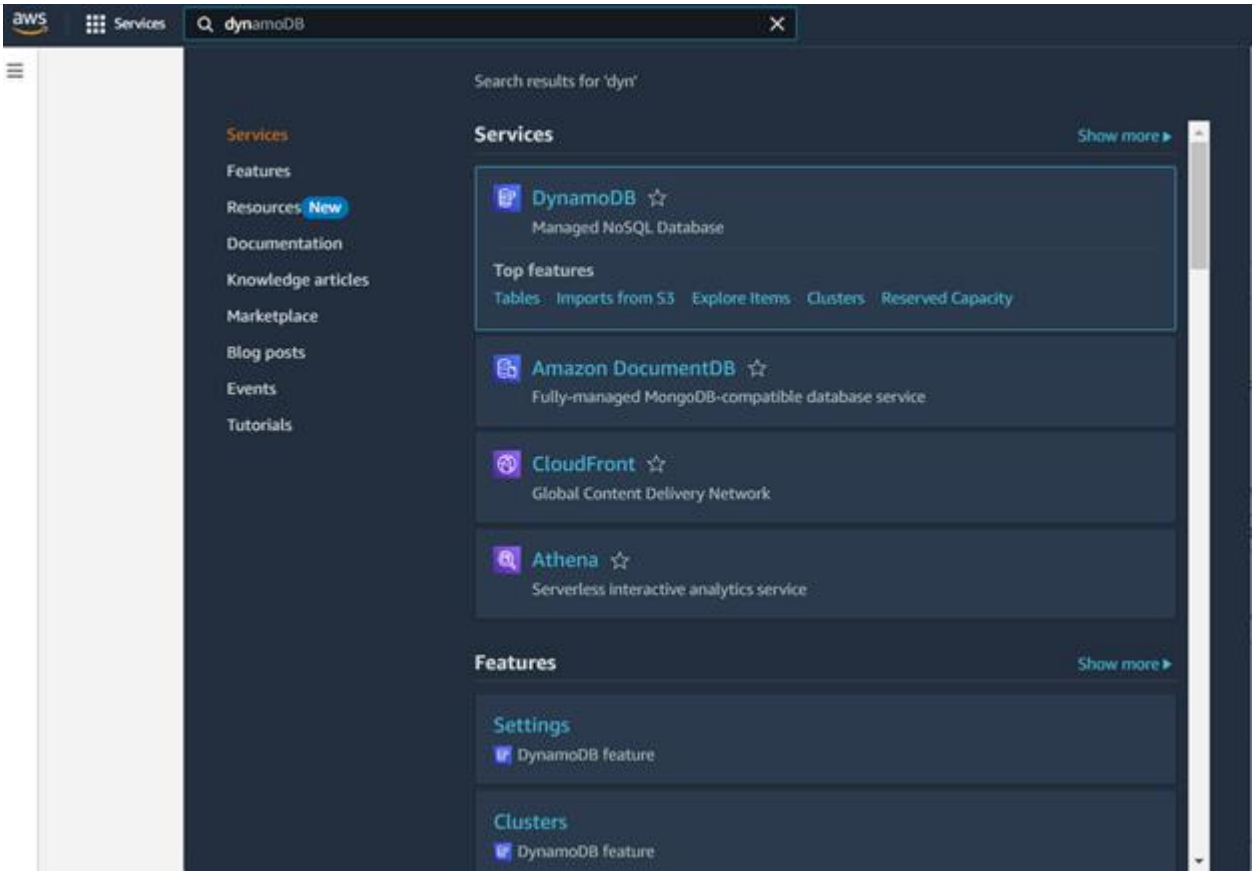
- Log in to the AWS Management Console
- After setting up your account, log in to the AWS Management Console.

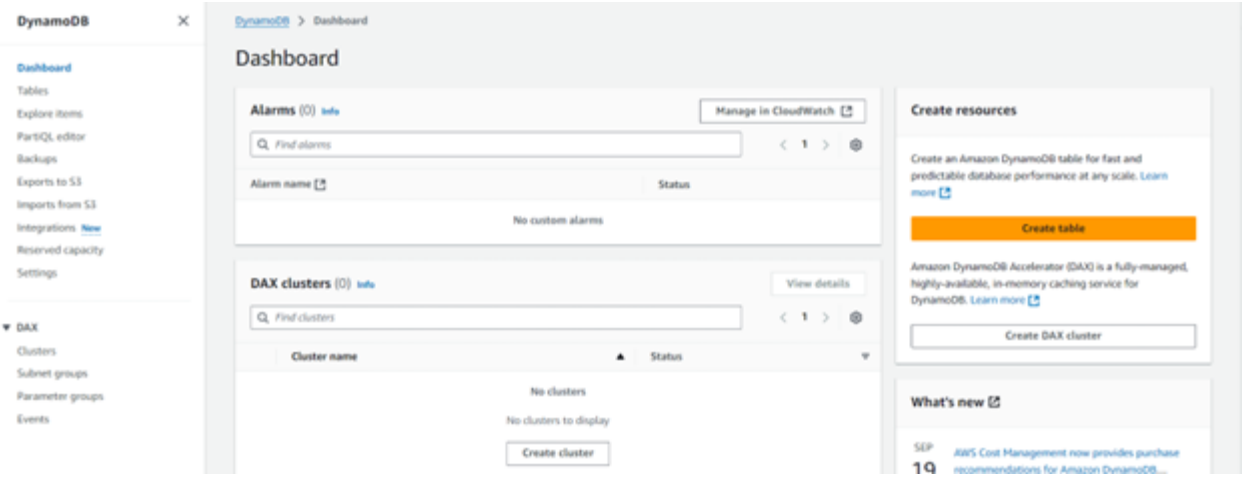


Milestone 3: DynamoDB Databases Creation and Setup

- Navigate to the DynamoDB

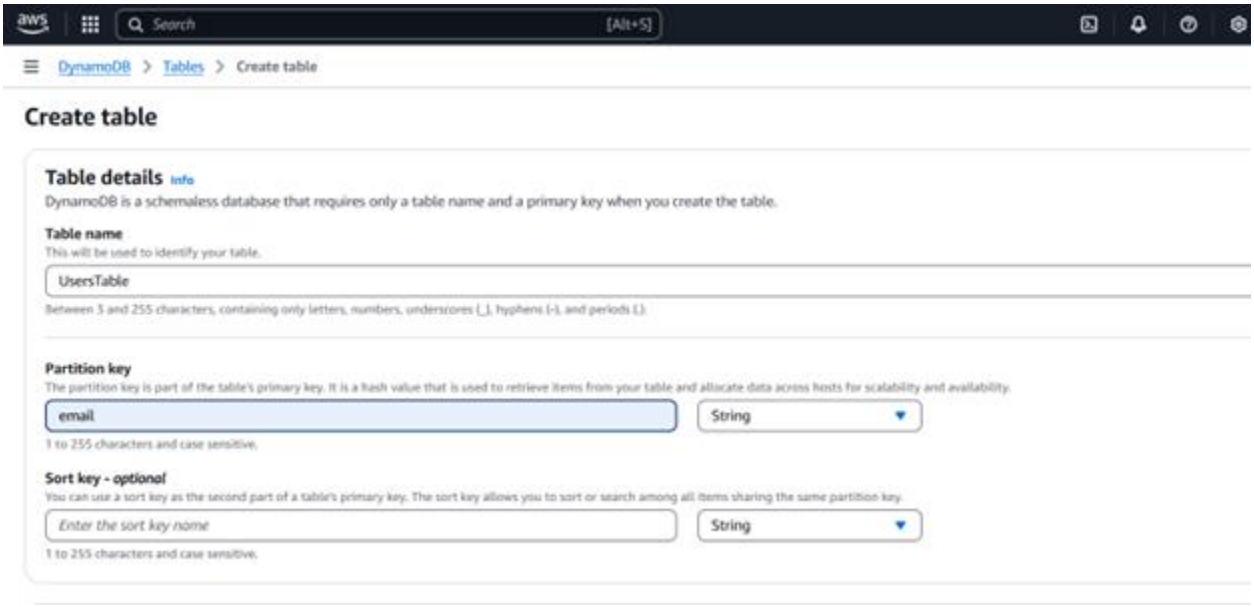
- In the AWS Console, navigate to DynamoDB and click on create tables.

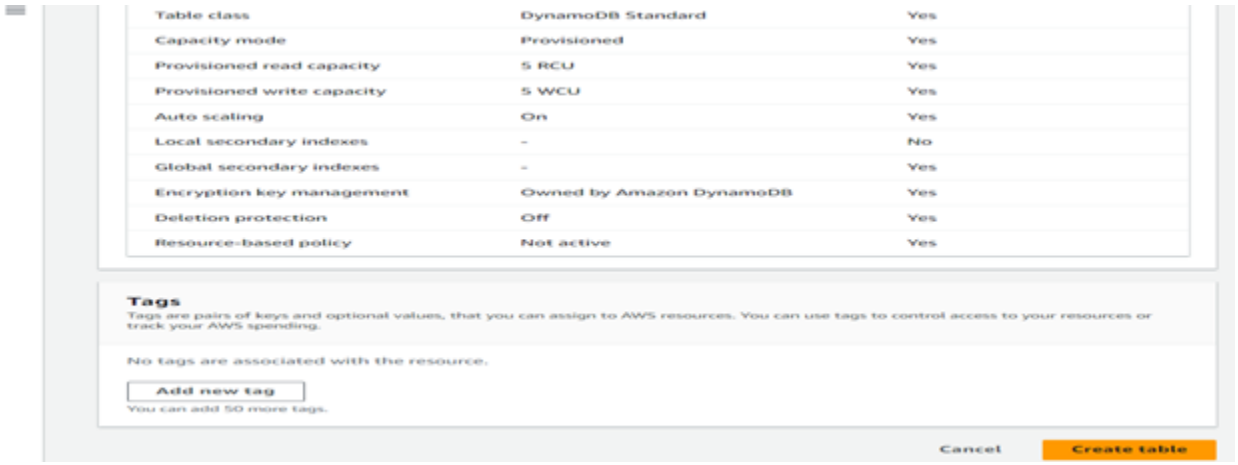




- Create a DynamoDB table for storing data

  Create Users Table with partition key "email" with type String and click on create tables.





Create Appointments Table with partition key "appointment _id" with type String and click on create tables.

aws | Search [Alt+S]

DynamoDB > Tables > Create table

Create table

Table details Info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

AppointmentsTable

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

appointment_id | String

1 to 255 characters and case sensitive.

Sort key - *optional*

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name | String

1 to 255 characters and case sensitive.

Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

Tags

Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

Add new tag

You can add 50 more tags.

Cancel | Create table

Tables (2/10) Info

Actions ▾ | Delete | Create table

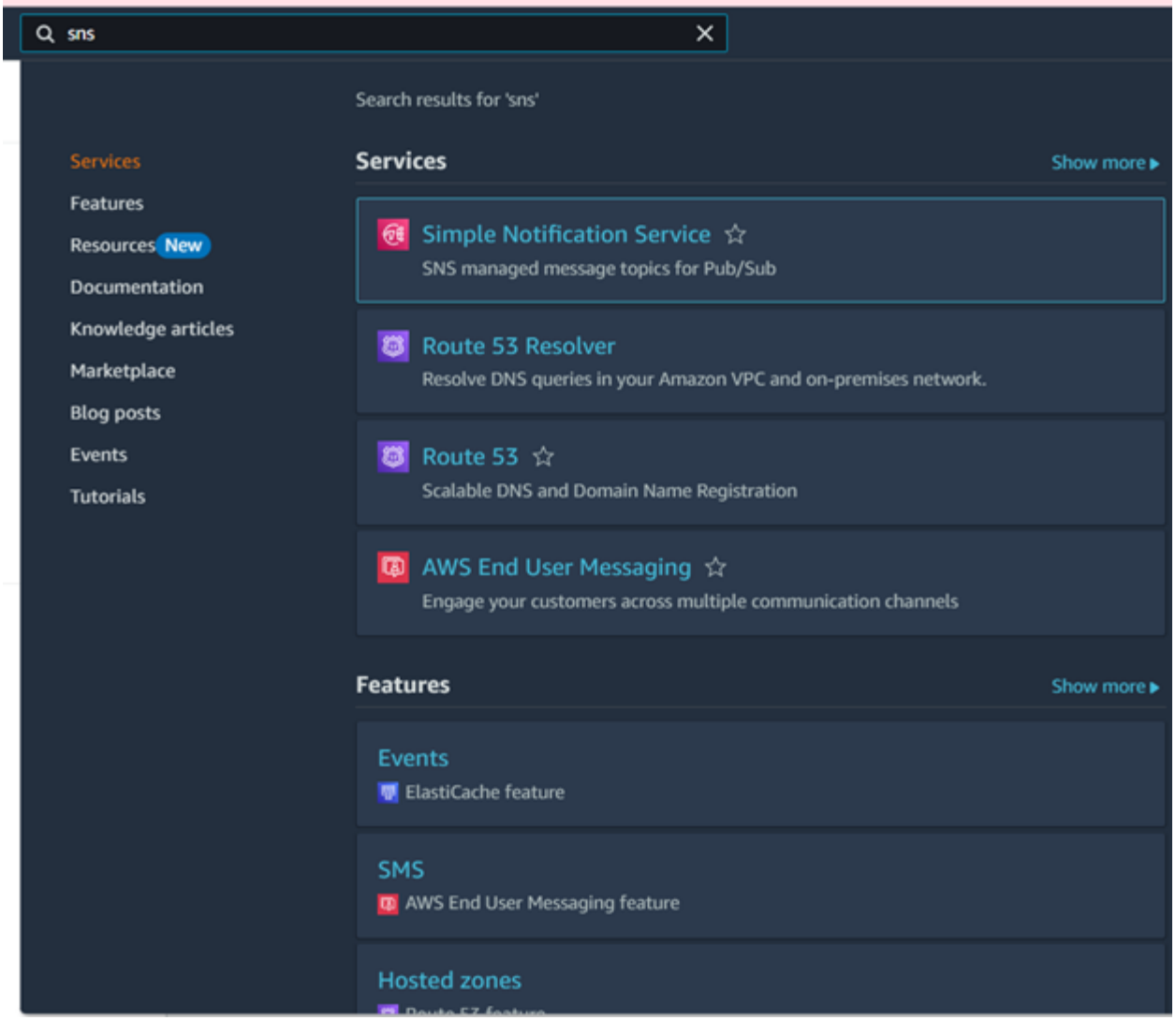
Status | Any tag key | Any tag value | 1

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection
AppointmentsTable	Active	appointment_id (S)	-	0	0	Off
NextGenHospital_Appointments	Active	appointment_id (S)	-	0	0	Off
NextGenHospital_ContactMessages	Active	message_id (S)	-	0	0	Off
NextGenHospital_Doctors	Active	doctor_id (S)	-	0	0	Off
NextGenHospital_PatientRecords	Active	patient_id (S)	-	0	0	Off
NextGenHospital_Users	Active	email (S)	-	0	0	Off
SalonAppointments	Active	appointment_id (S)	user_email (S)	0	0	Off
SalonStylists	Active	stylist_id (S)	-	0	0	Off
SalonUsers	Active	email (S)	-	0	0	Off
UsersTable	Active	email (S)	-	0	0	Off

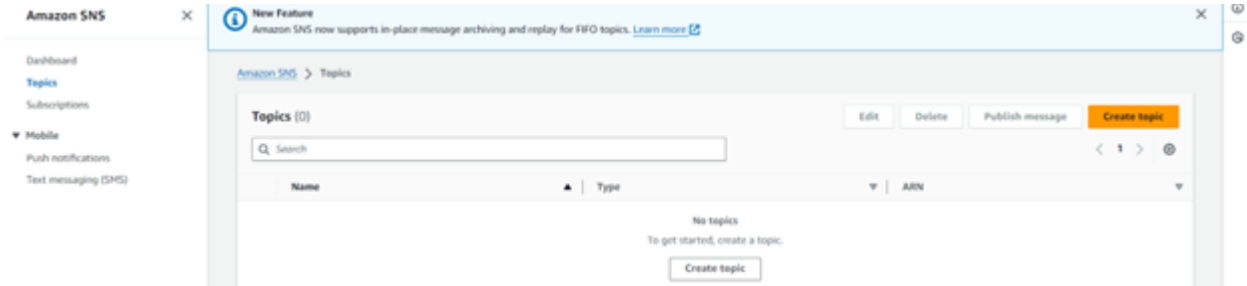
Milestone 4 : SNS Notification Setup

Create SNS topics for sending email notifications to users

- In the AWS Console, search for SNS and navigate to the SNS Dashboard.

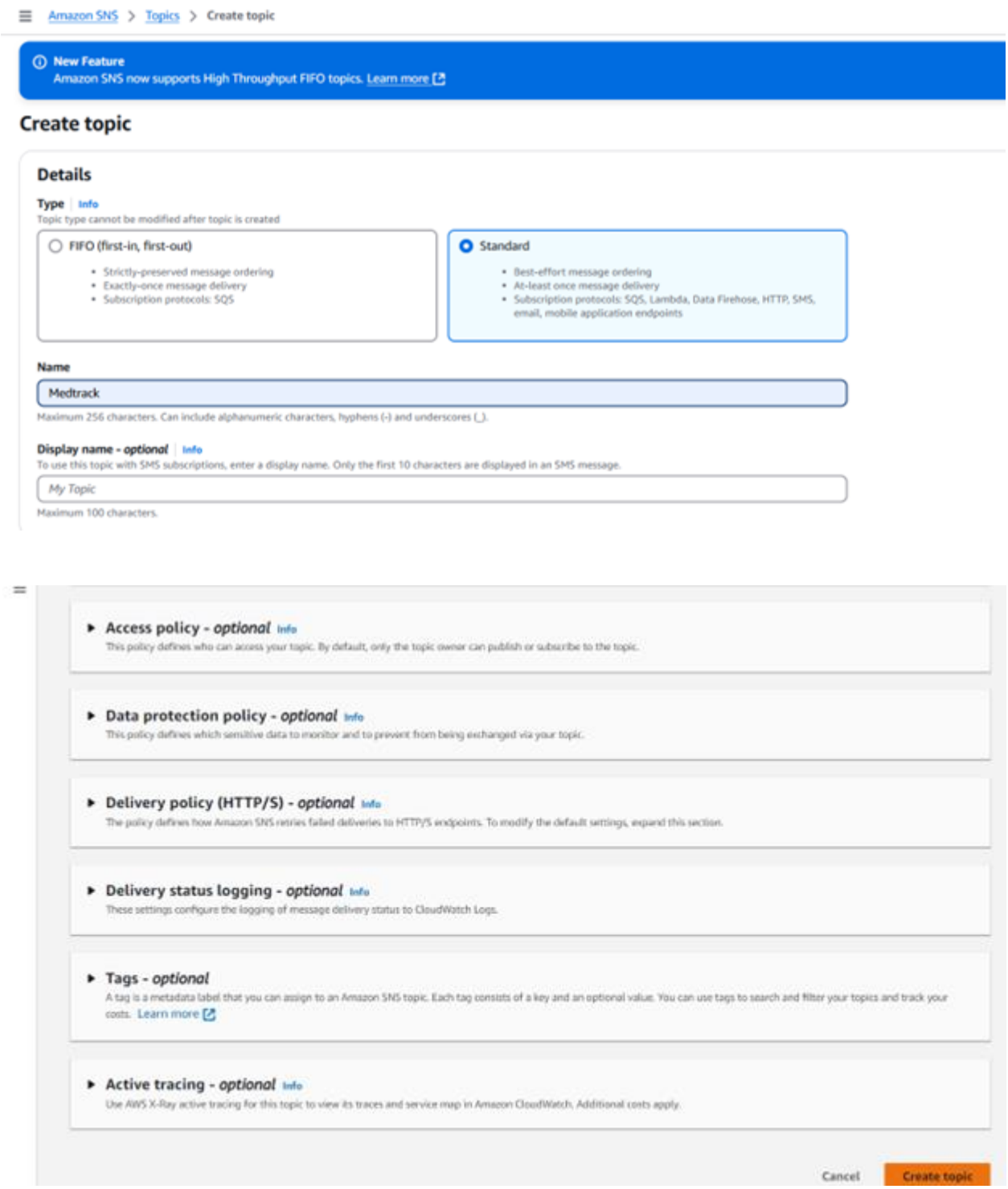


- Click on **Create Topic** and choose a name for the topic.

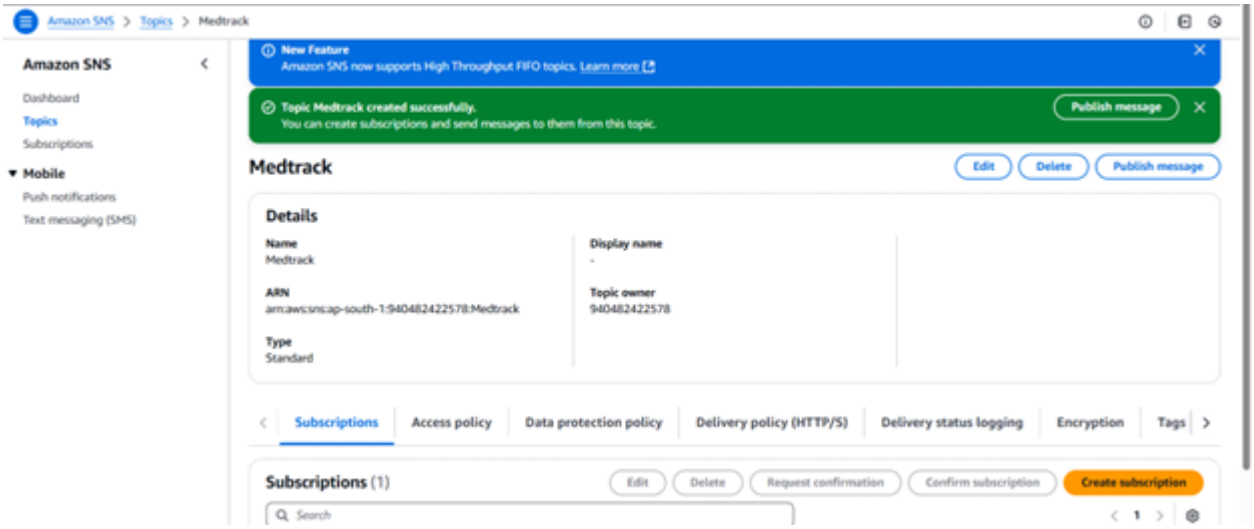


- Add image

  Choose Standard type for general notification use cases and Click on Create Topic.

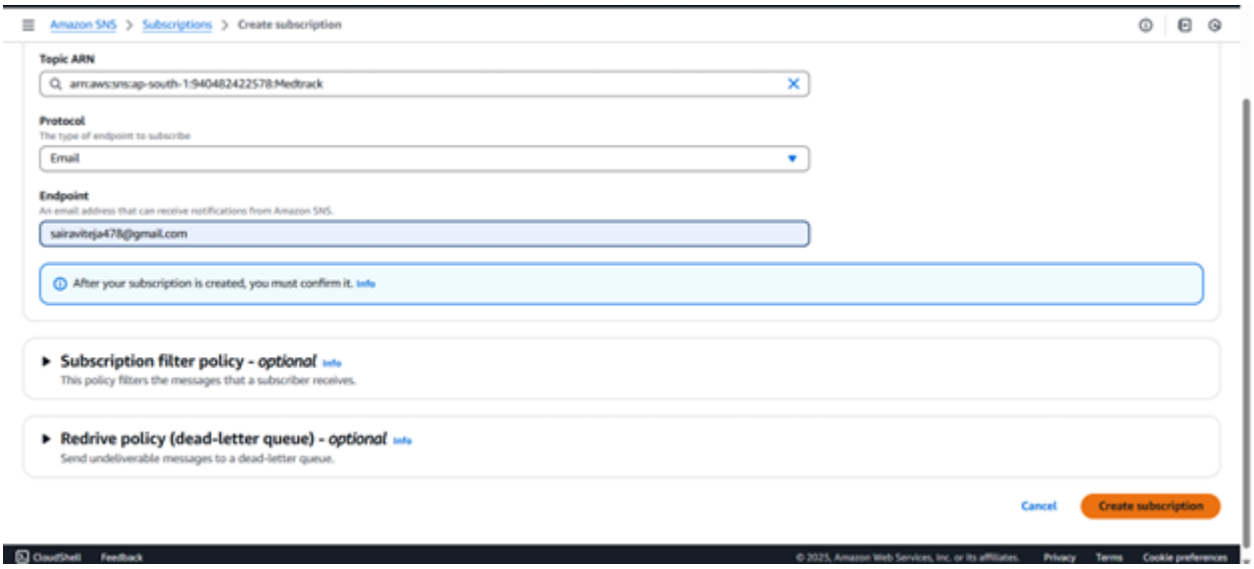


- Configure the SNS topic and note down the **Topic ARN**.

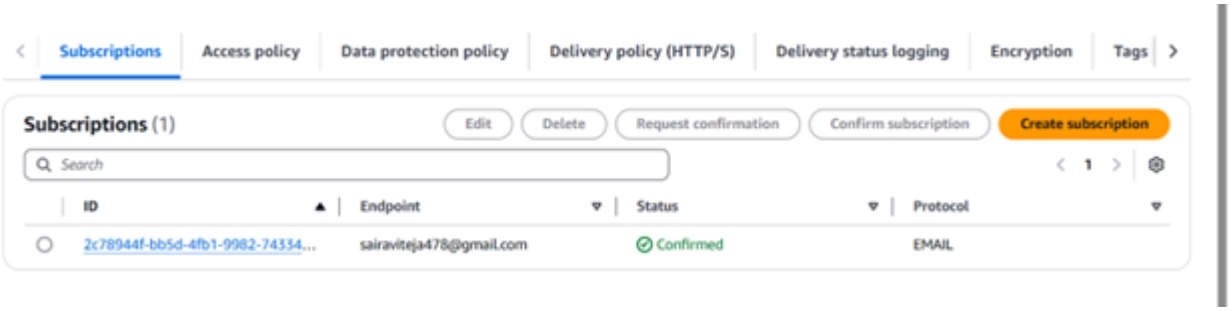


Subscribe Users and Admin

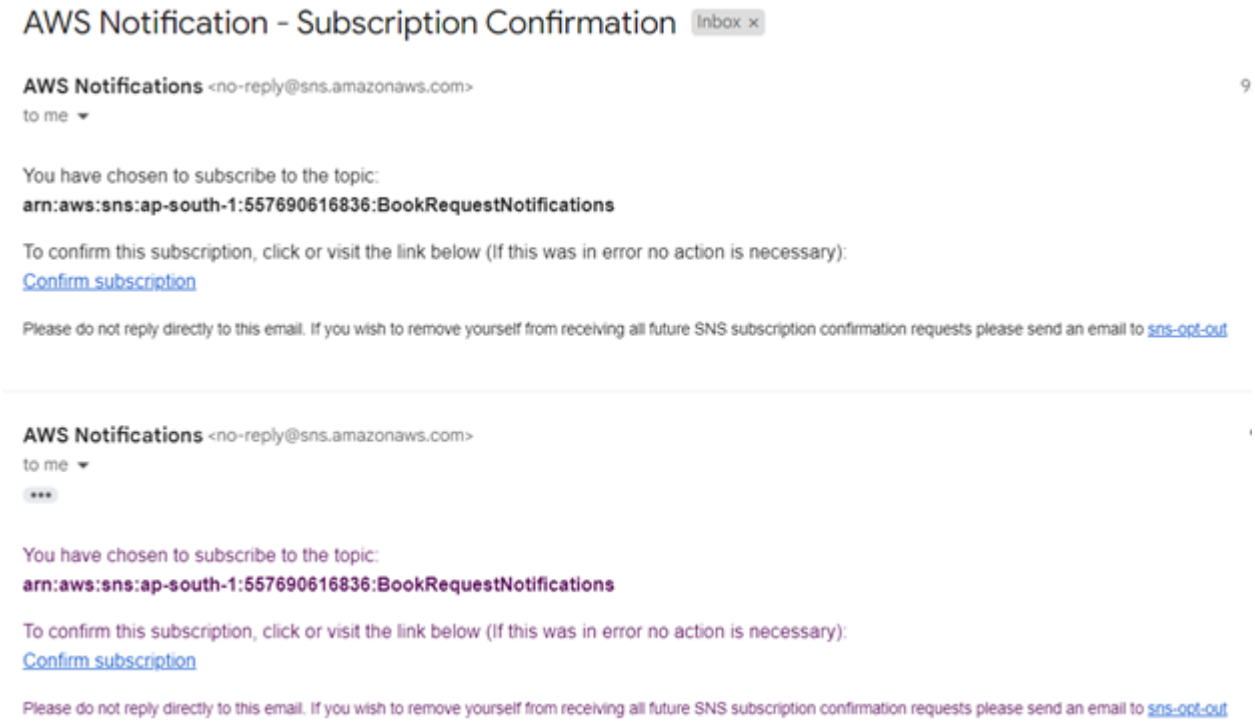
- Subscribe users (or admin staff) to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails



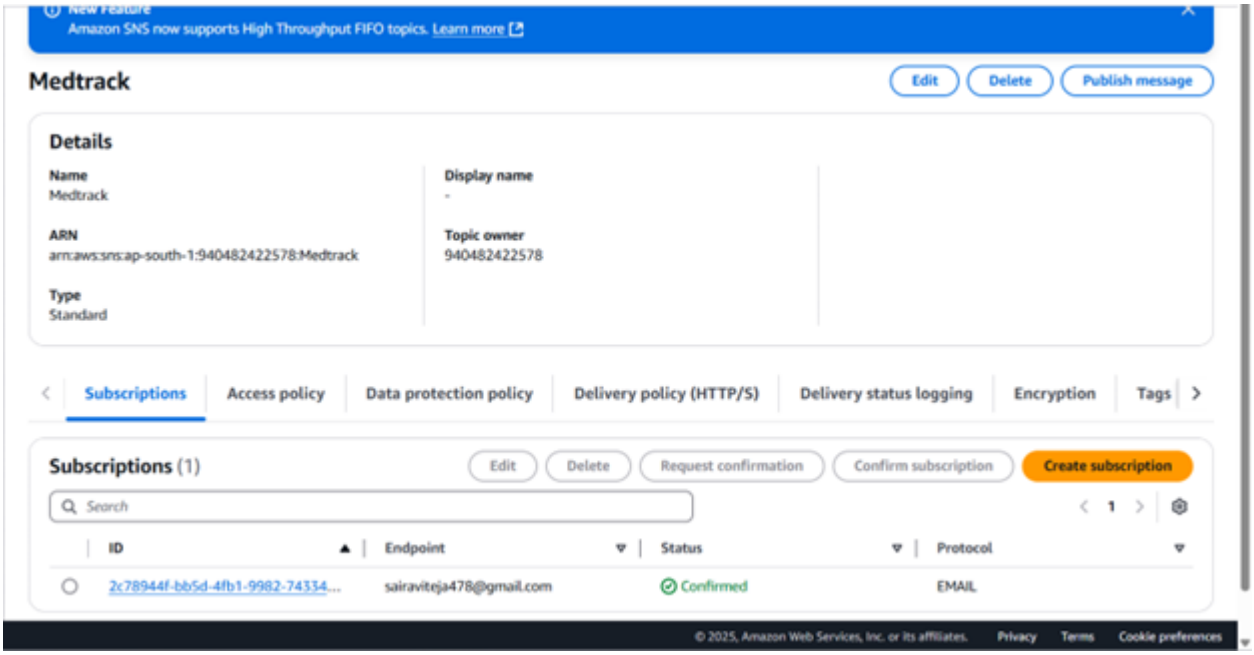
- After subscription request for the mail confirmation

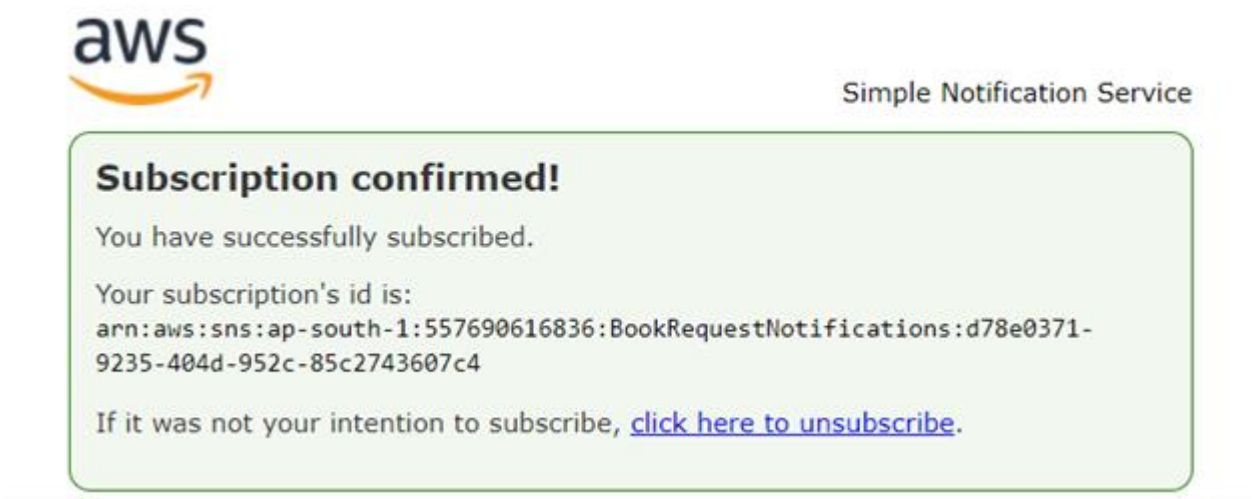


- Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.



- Successfully done with the SNS mail subscription and setup, now store the ARN link.

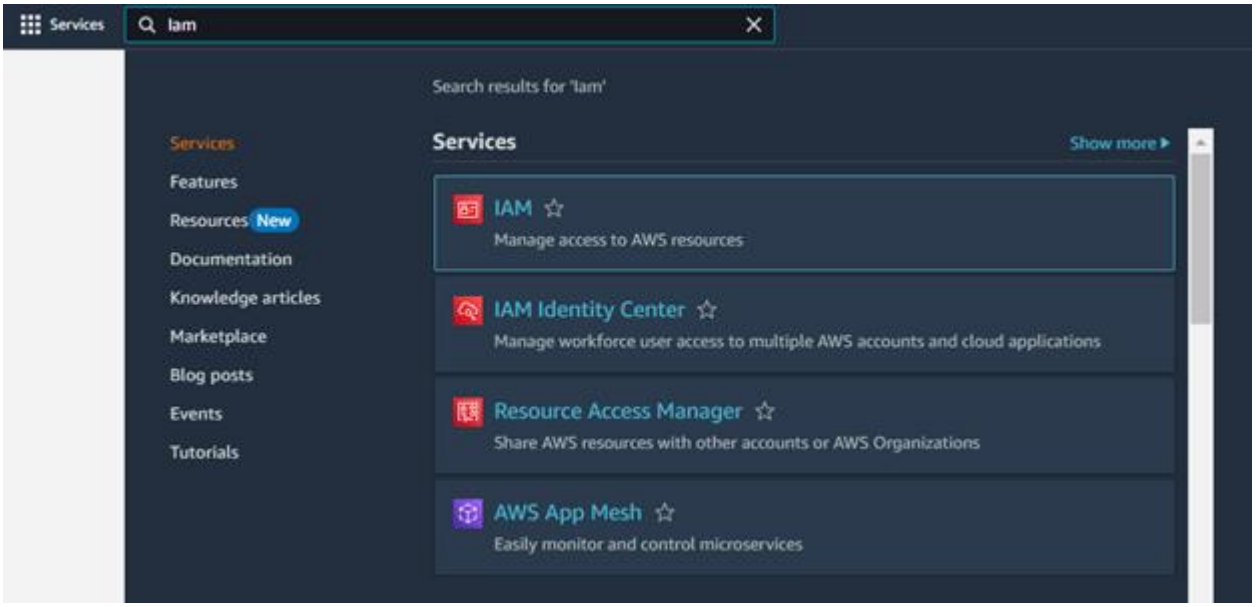


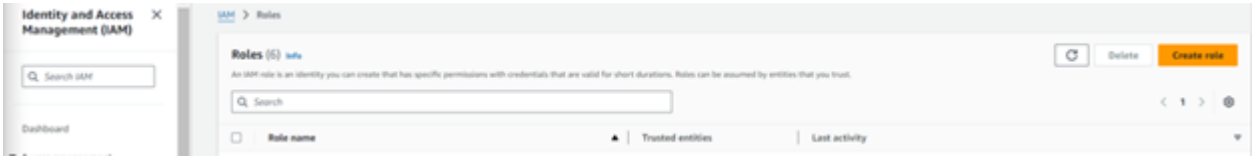


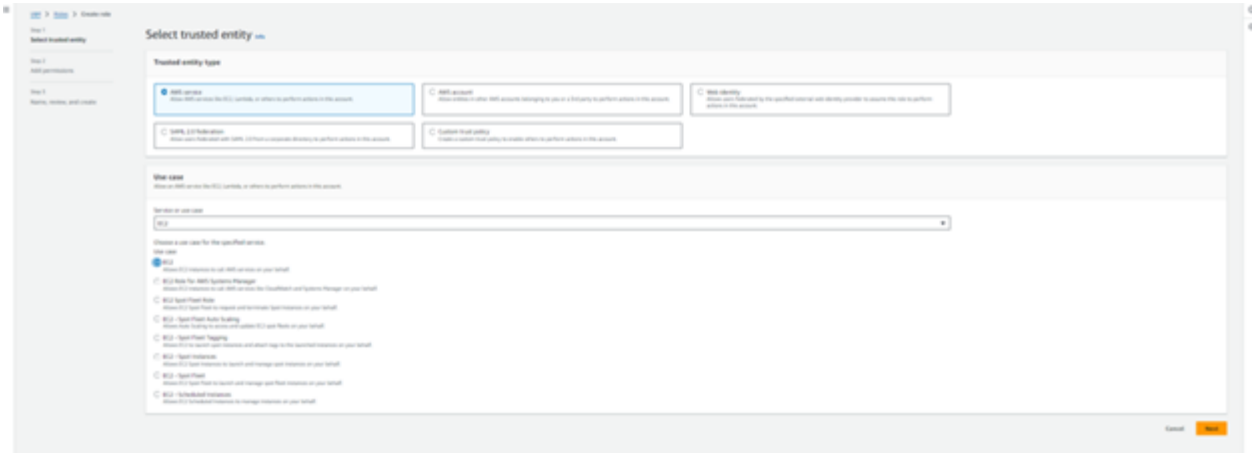
Milestone 5: IAM Role Setup

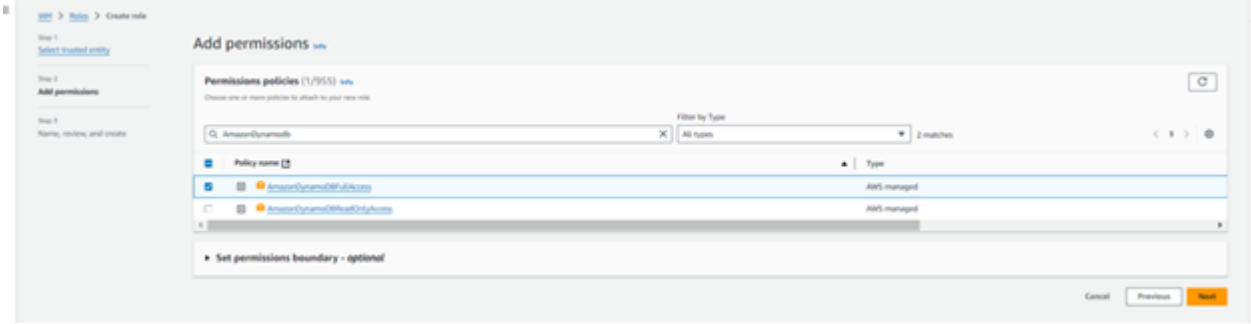
Create IAM Role

- In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS.


Services
Iam
Search results for 'Iam'
Services
Features
Resources New
Documentation
Knowledge articles
Marketplace
Blog posts
Events
Tutorials
Services
Show more
IAM
Manage access to AWS resources
IAM Identity Center
Manage workforce user access to multiple AWS accounts and cloud applications
Resource Access Manager
Share AWS resources with other accounts or AWS Organizations
AWS App Mesh
Easily monitor and control microservices


Identity and Access Management (IAM)
Dashboard
IAM > Roles
Roles
Delete
Create role
Role name
Trusted entities
Last activity

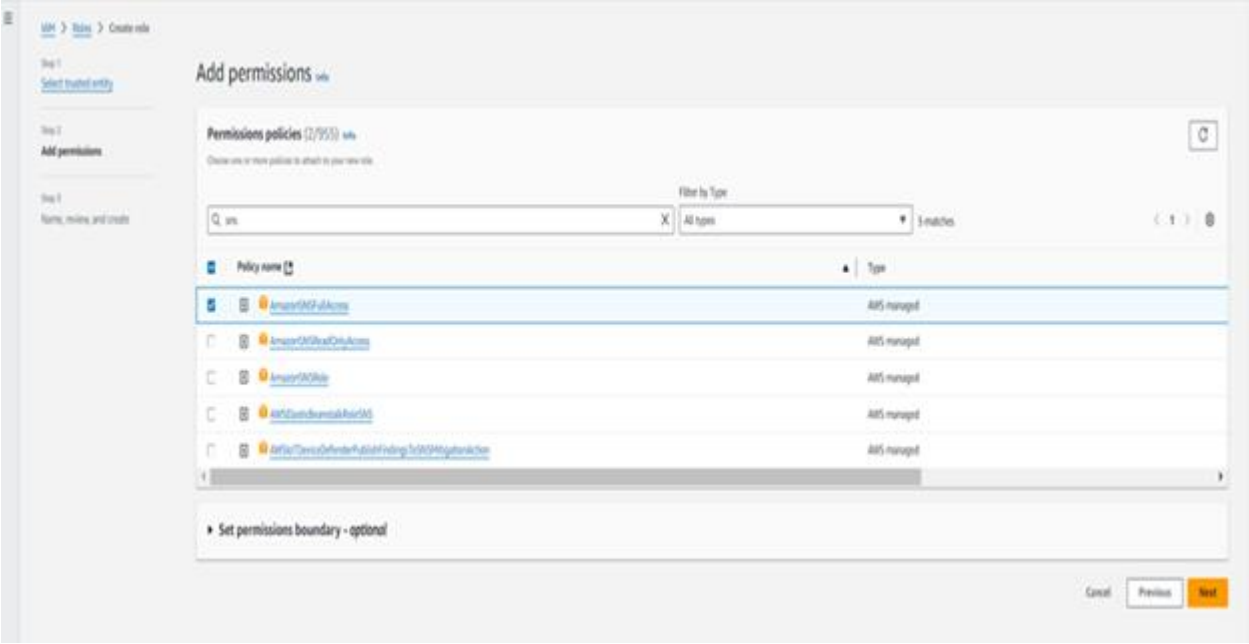

Select trusted entity



- Attach Policies

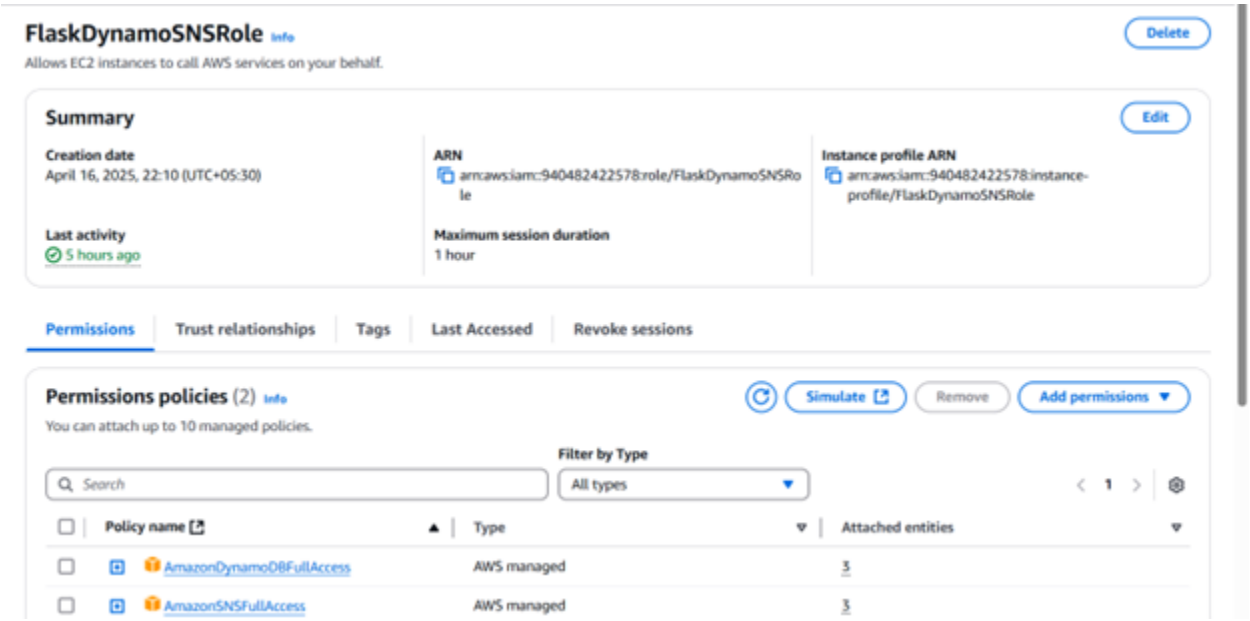
  Attach the following policies to the role:
- AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.

- AmazonSNSFullAccess: Grants EC2 the ability to send notifications via SNS.



- To create a role named **flask dynamo db sns,** go to the AWS IAM console, create a new role, and assign Dynamo DB Full Access and SNS Full Access policies. Name the role flask dynamodb sns

and attach it to the necessary AWS services. This role will allow your Flask app to interact with both DynamoDB and SNS seamlessly.

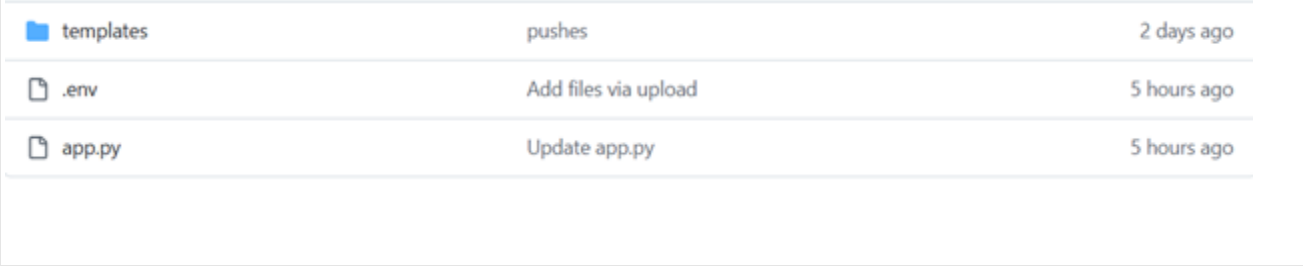


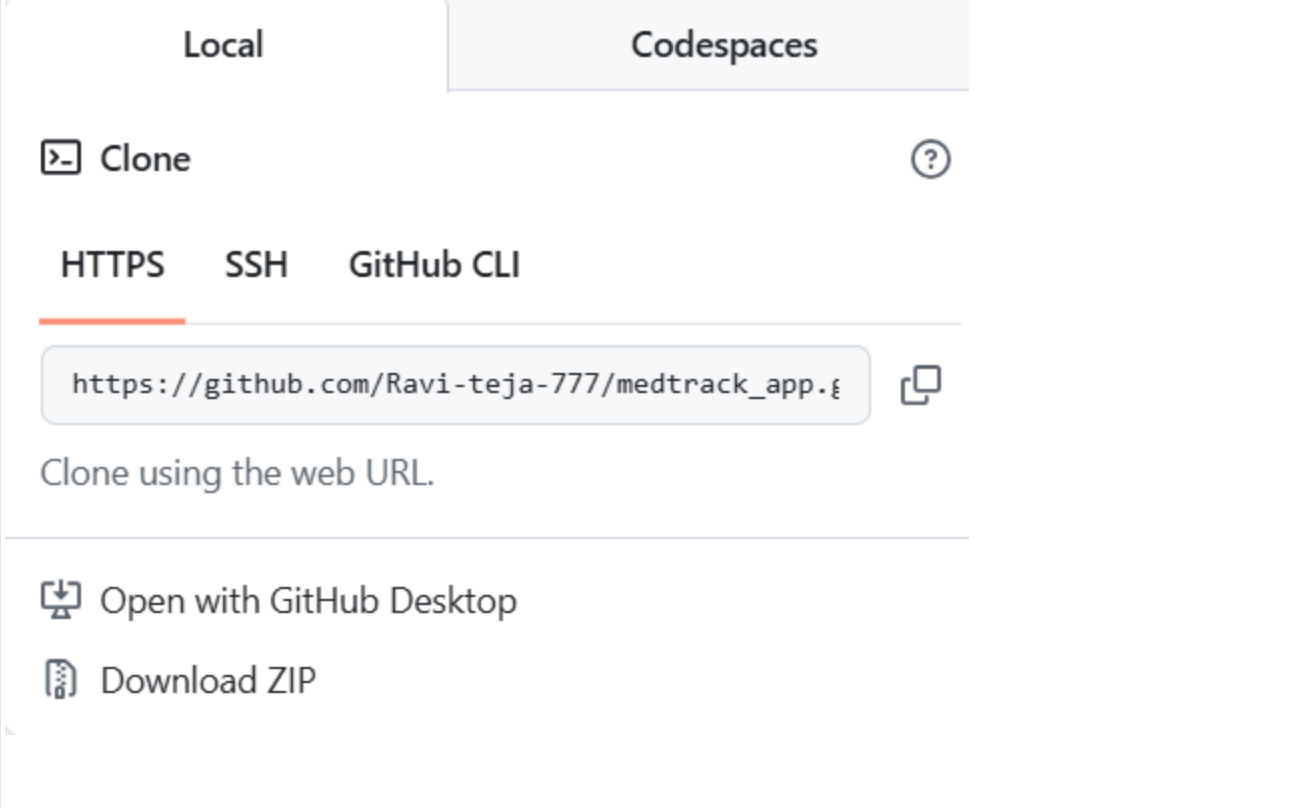
Milestone 6: EC2 Instances setup

Load your Project Files to GitHub

- Note: Load your Flask app and Html files into GitHub repository.

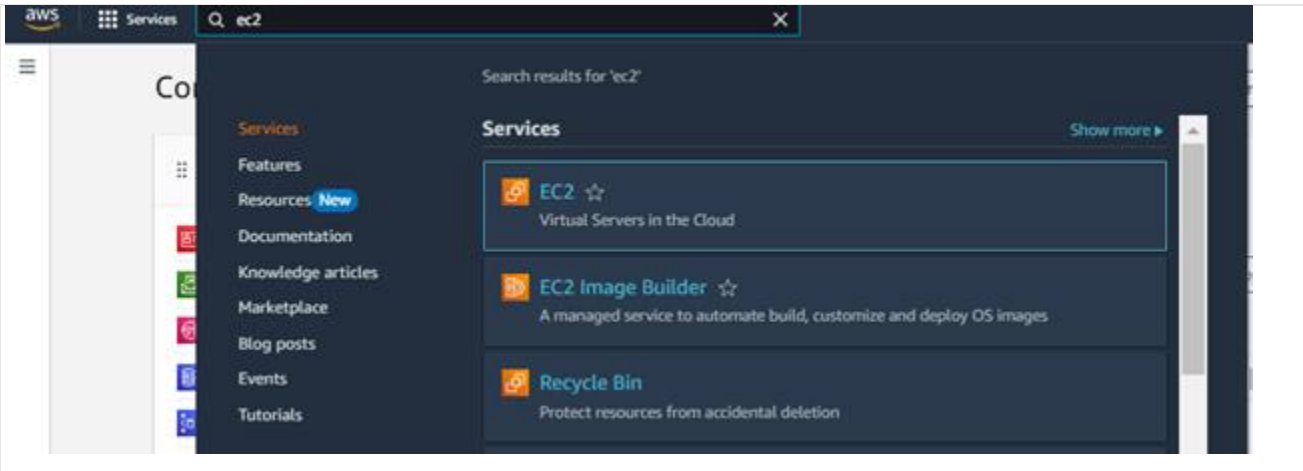
- To upload your Flask app and HTML files to GitHub, initialize the repository using git in it, commit the files with git add . && git commit -m "Initial commit", and push them using git push -u origin main. This will load your project onto GitHub.





Launch an EC2 instance to host the Flask

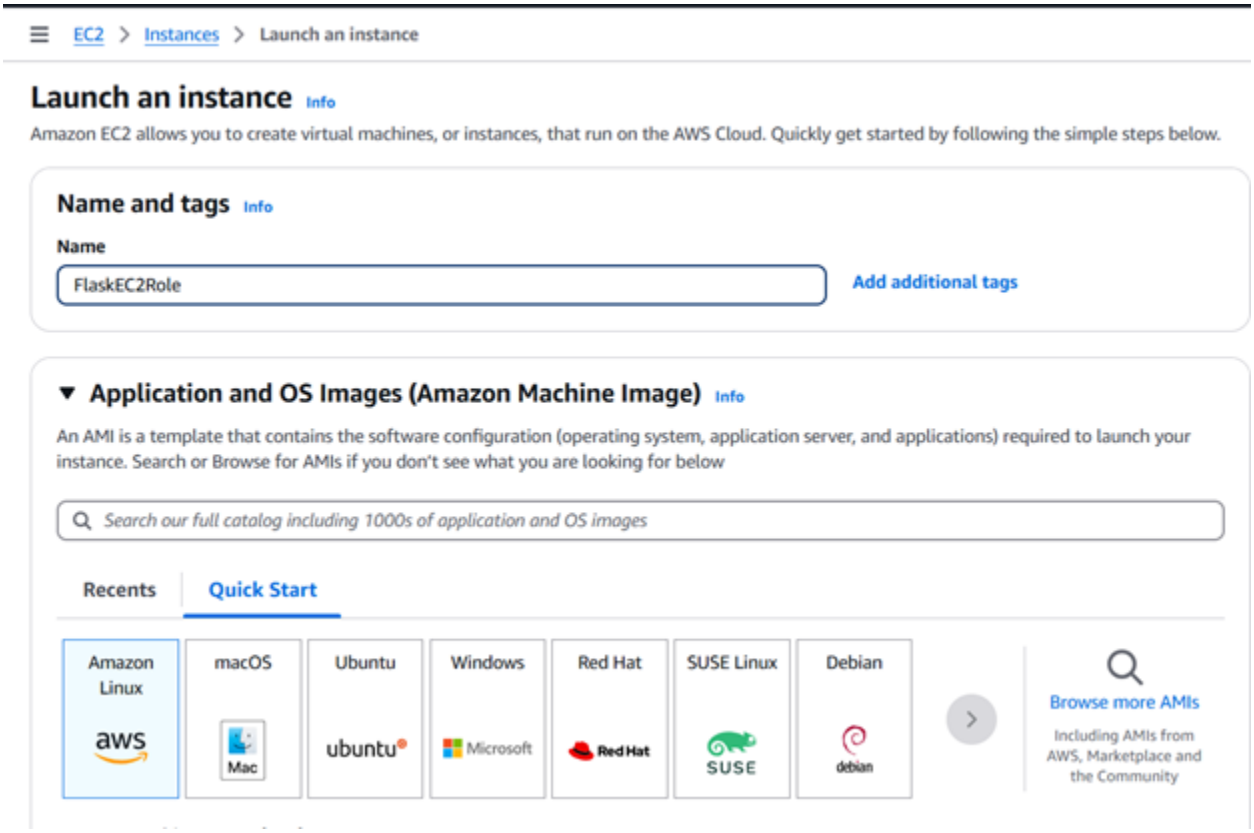
- In the AWS Console, navigate to EC2 and launch a new instance.



To launch an EC2 instance with the name flaskec2role, follow these steps:

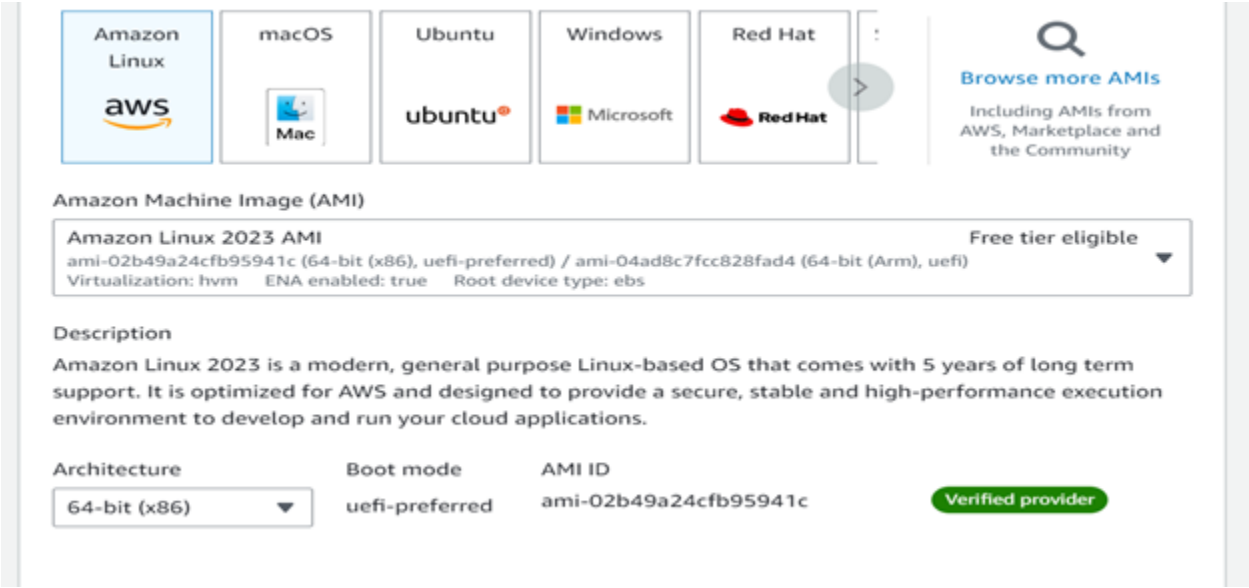
- Go to the AWS EC2 Dashboard and click on Launch Instance.

- Select your desired AMI, instance type, configure instance details, and under IAM role, choose the role flaskec2role. Finally, launch the instance.



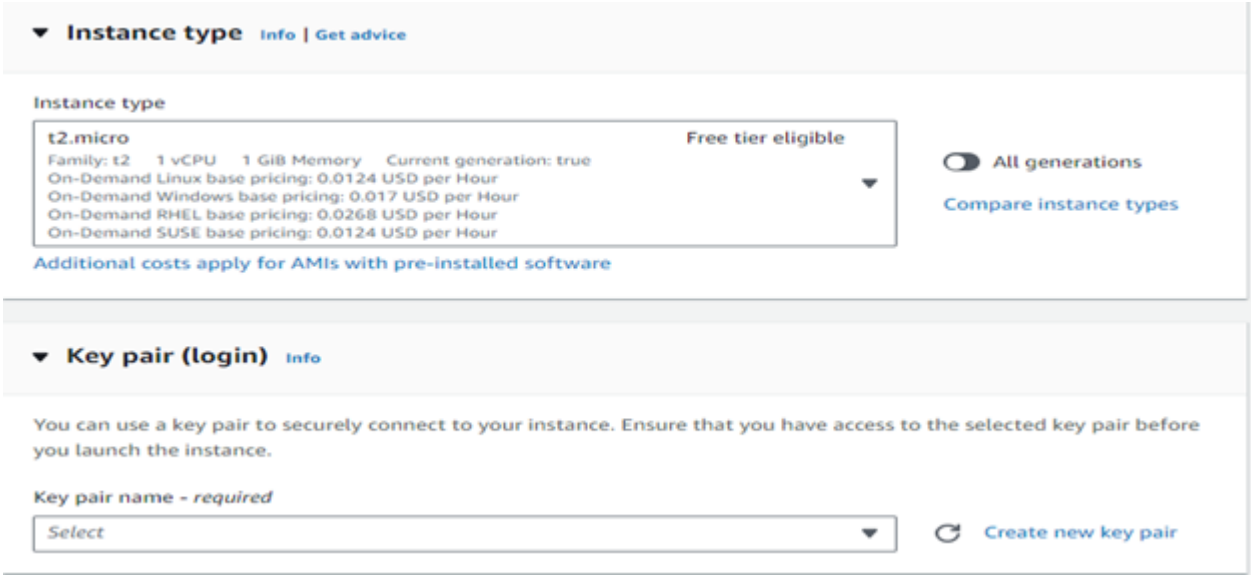
To launch an EC2 instance with Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instance type (free-tier eligible):

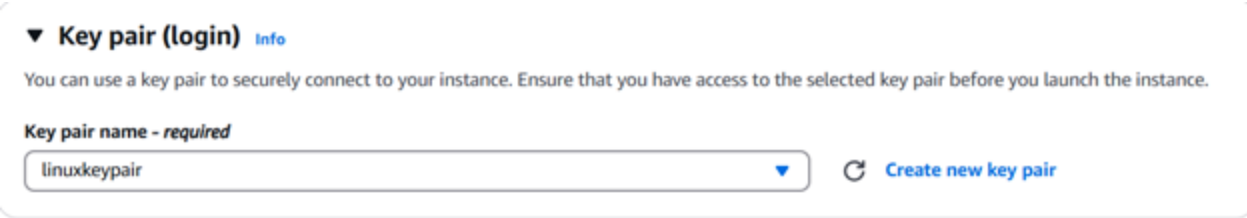
- In the Launch Instance wizard, choose Amazon Linux 2 or Ubuntu from the available AMIs.
- Select t2.micro as the instance type, which is free-tier eligible, and continue with the configuration and launch steps.

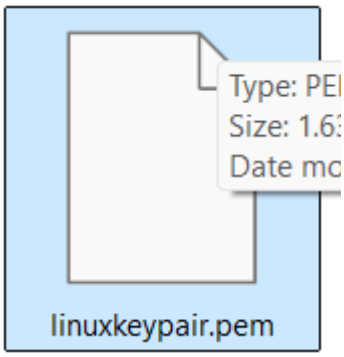


To create and download the key pair for server access:

- In the Launch Instance wizard, under the Key Pair section, click Create a new key pair.
- Name your key pair (e.g., flask keypair) and click Download Key Pair. This will download the .pem file to your system, which you will use to access the EC2 instance securely via SSH.







Configure security groups for HTTP, And SSH Access.

For network settings during EC2 instance launch:

- In the Network Settings section, select the VPC and Subnet you wish to use (if unsure, the default VPC and subnet should work).
- Ensure Auto-assign Public IP is enabled so your instance can be accessed from the internet.
- In Security Group, either select an existing one or create a new one that allows SSH (port 22) access to your EC2 instance for remote login.

Network settings Info

VPC - *required* Info

vpc-03cdc7b6f19dd7211 (default)
172.31.0.0/16

Subnet Info

No preference — Create new subnet

Auto-assign public IP Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

- (•) Create security group
- ( ) Select existing security group

Security group name - *required*

launch-wizard

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and ._-:/()#,@[]+=&;{}!$*

Description - *required* Info

launch-wizard created 2024-10-13T17:49:56.622Z

Inbound Security Group Rules

Security group rule 1 (TCP, 22, 0.0.0.0/0) — Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22

Source type Info	Source Info	Description - *optional* Info
Anywhere	Add CIDR, prefix list or security 0.0.0.0/0	e.g. SSH for admin desktop

Security group rule 2 (TCP, 80, 0.0.0.0/0) — Remove

Type Info	Protocol Info	Port range Info
HTTP	TCP	80

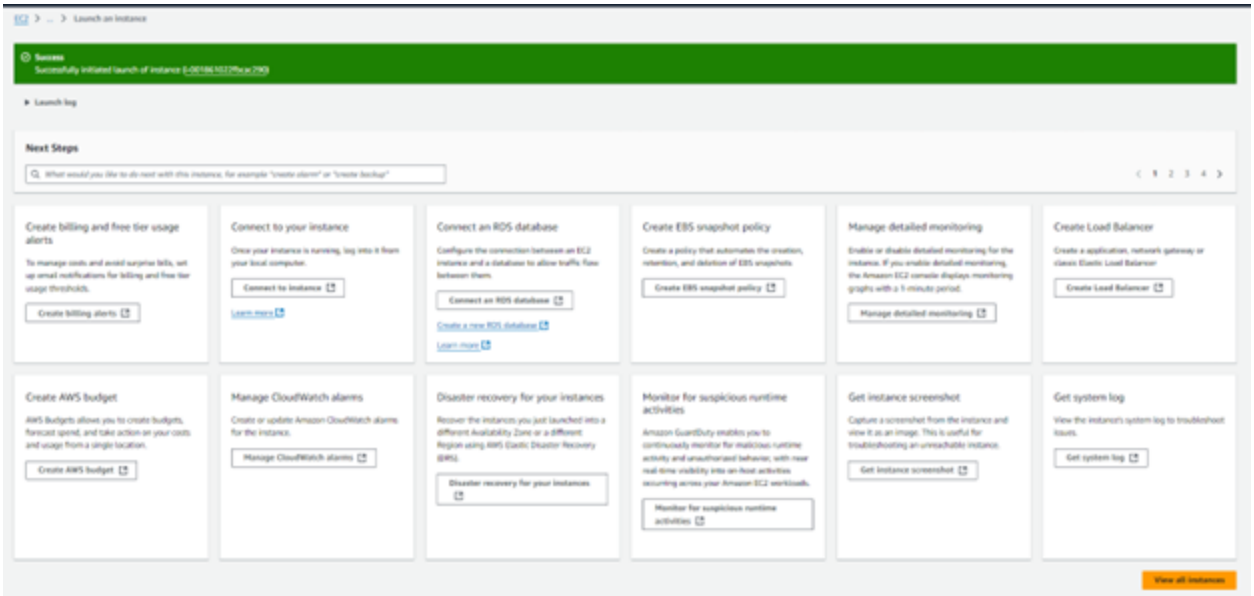
Source type Info	Source Info	Description - *optional* Info
Custom	Add CIDR, prefix list or security 0.0.0.0/0	e.g. SSH for admin desktop

Security group rule 3 (TCP, 5000, 0.0.0.0/0) — Remove

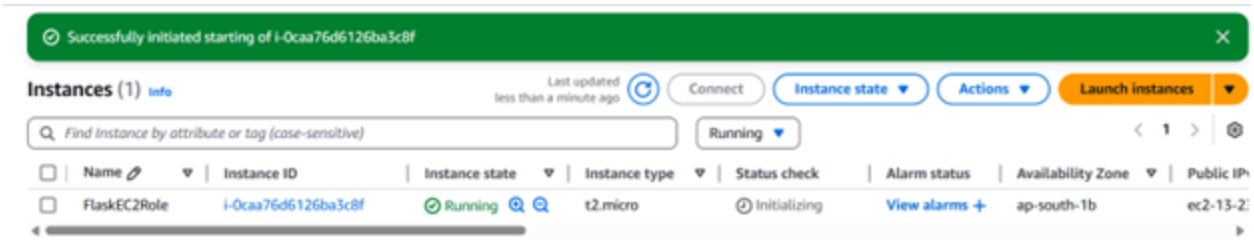
Type Info	Protocol Info	Port range Info
Custom TCP	TCP	5000

Source type Info	Source Info	Description - *optional* Info
Custom	Add CIDR, prefix list or security 0.0.0.0/0	e.g. SSH for admin desktop

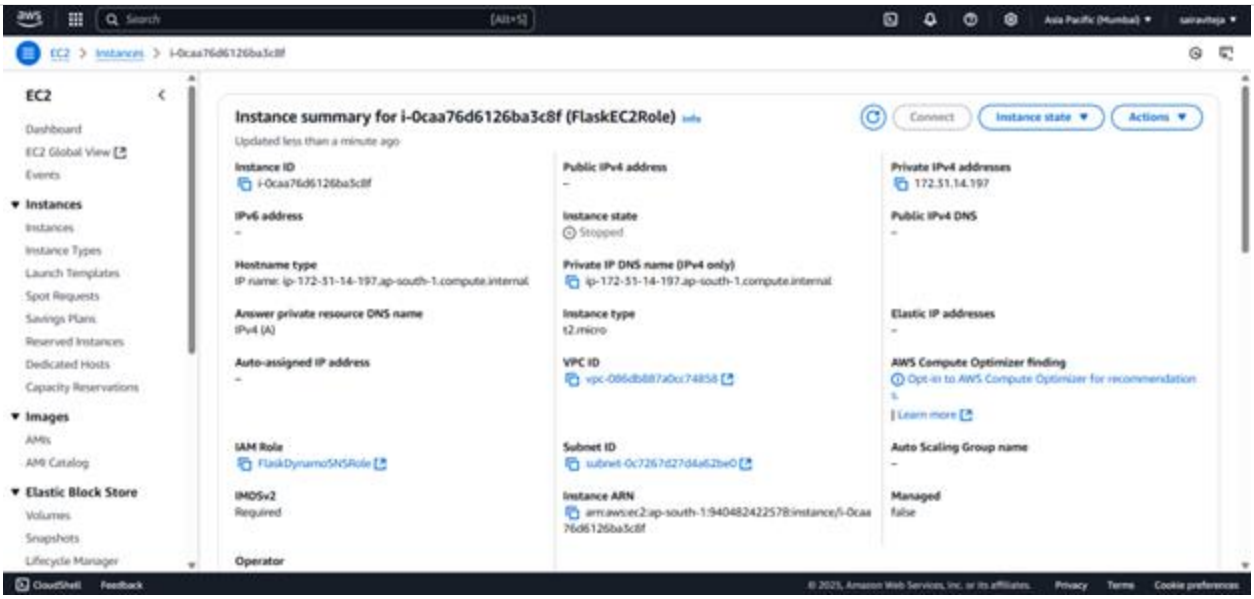
Add security group rule



- To connect to EC2 using EC2 Instance Connect, start by ensuring that an IAM role is attached to your EC2 instance. You can do this by selecting your instance, clicking on Actions, then navigating to Security and selecting Modify IAM Role to attach the appropriate role. After the IAM role is connected, navigate to the EC2 section in the AWS Management Console. Select the EC2 instance you wish to connect to. At the top of the EC2 Dashboard, click the Connect button. From the connection methods presented, choose EC2 Instance Connect. Finally, click Connect again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.

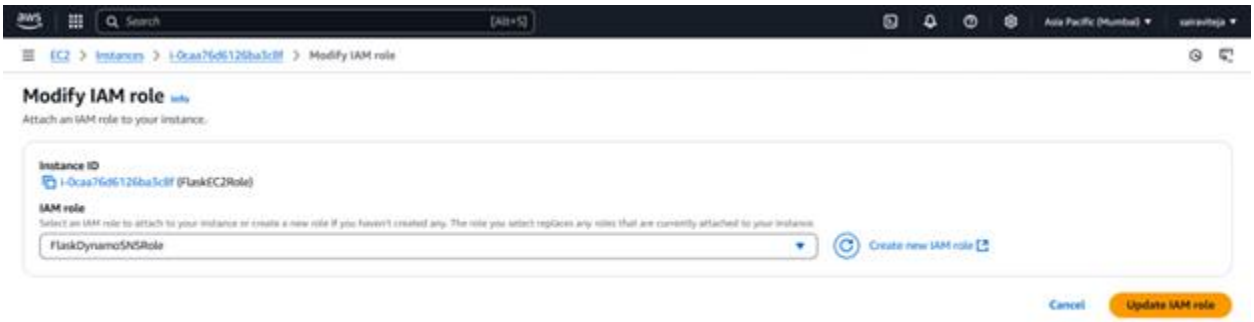


- The EC2 instance you are launching is configured with Amazon Linux 2 or Ubuntu as the AMI, t2.micro as the instance type (free-tier eligible), and flaskec2role IAM role for appropriate permissions. The flaskkeypair key pair is created for secure server access



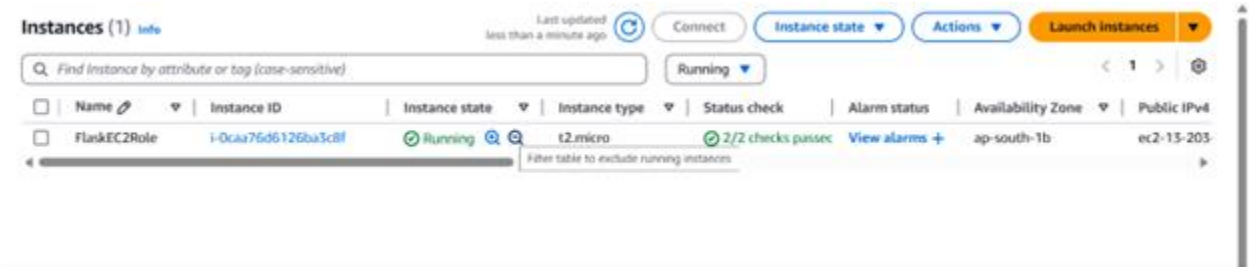
To modify the **IAM role** for your EC2 instance:

- Go to the **AWS IAM Console**, select **Roles**, and find the **flaskec2role**.
- Click **Attach Policies**, then choose the required policies (e.g., **DynamoDB Full Access**, **SNS Full Access**) and click **Attach Policy**.
- If needed, update the instance to use this modified role by selecting the EC2 instance, clicking **Actions**, then **Security**, and **Modify IAM role** to select the updated role.

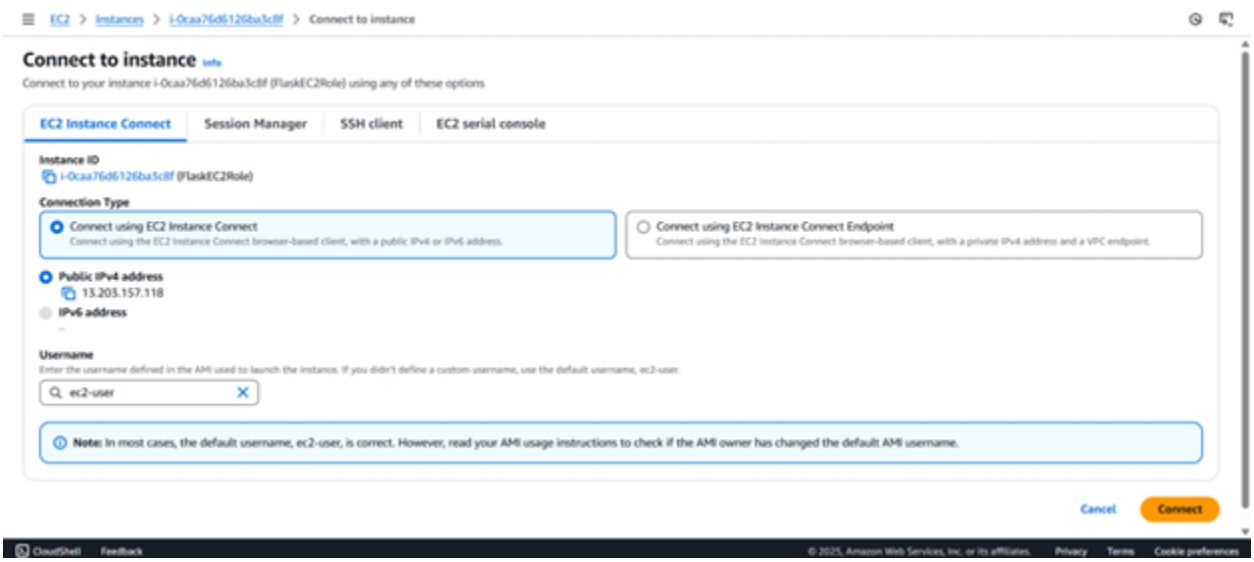


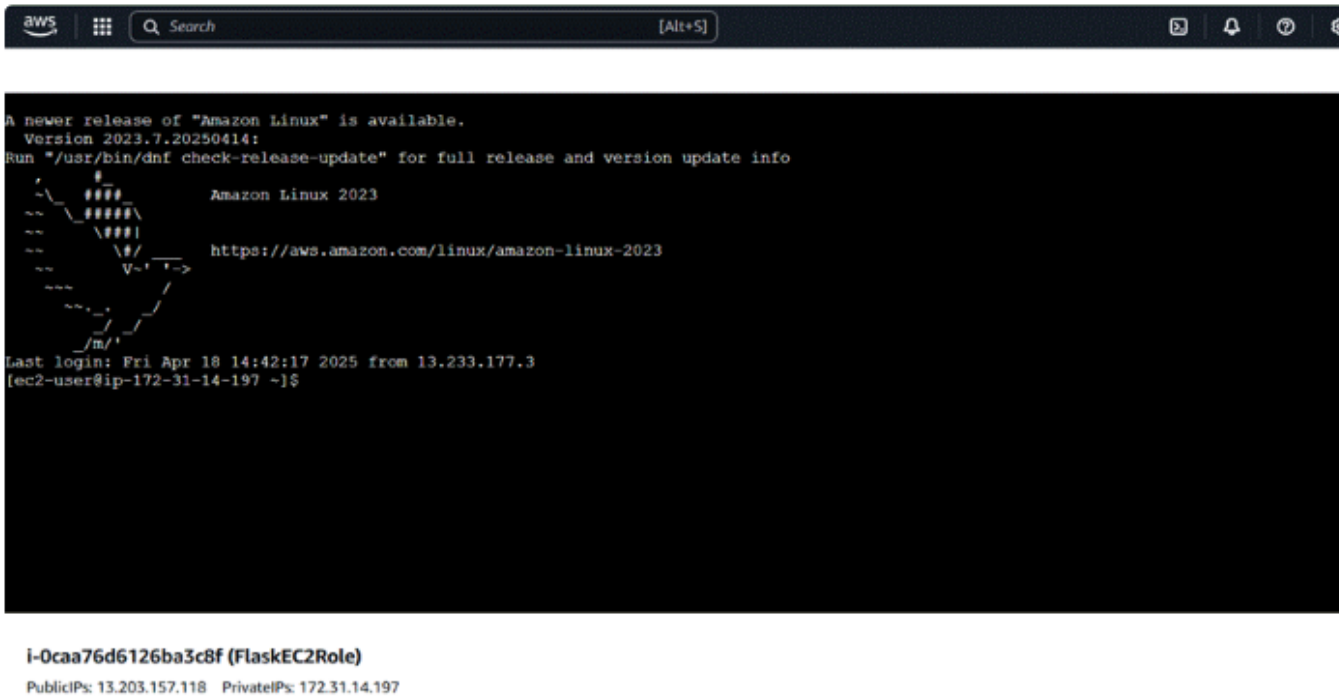
To connect to your EC2 instance:

- Go to the EC2 Dashboard, select your running instance, and click Connect.
- Follow the instructions provided in the Connect To Your Instance dialog, which will show the SSH command (e.g., ssh -i flask keypair .pem ec2-user@<public- ip>) to access your instance using the downloaded .pem key.



- Now connect the EC2 with the files





i-0caa76d6126ba3c8f (FlaskEC2Role)

PublicIPs: 13.203.157.118 PrivateIPs: 172.31.14.197

Milestone 7: Deployment Using EC2

- Install Software on the EC2 Instance

Install Python3, Flask, and Git

- On Amazon Linux 2:

```
sudo yum update -y
sudo yum install python3 git
sudo pip3 install flask boto3
```

- Verify Installations:

```
flask --version
git –version
```

Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

- Run: ‘git clone https://github.com/your-github-username/your-repository-name.git’
- Note: change your-github-username and your-repository-name with your credentials

here: ‘git clone https://github.com/Ravi-teja-777/medtrack_app.git

- This will download your project to the EC2 instance.

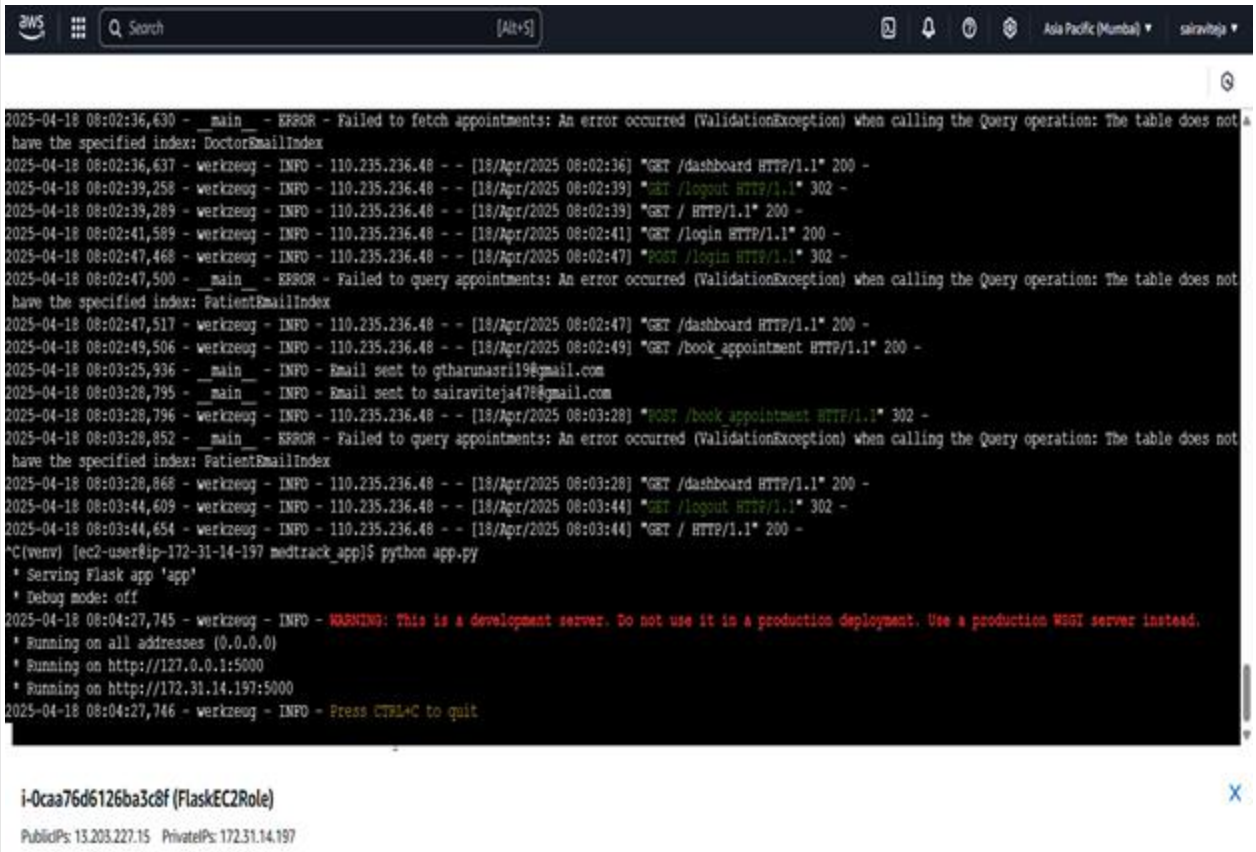
To navigate to the project directory, run the following command:

```
cd Medtrack
```

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

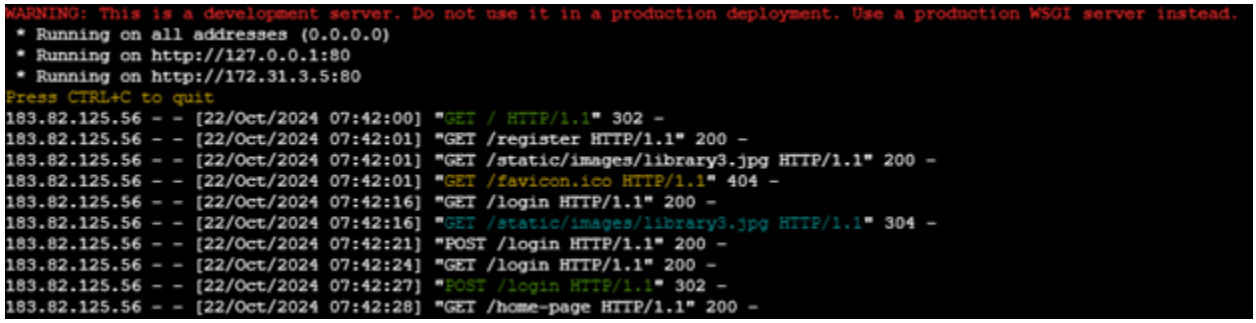
sudo flask run --host=0.0.0.0 --port=5000



Verify the Flask app is running:

http://your-ec2-public-ip

- Run the Flask app on the EC2 instance



Access the website through:

PublicIPs: https://13.201

Milestone 8: Testing and Deployment

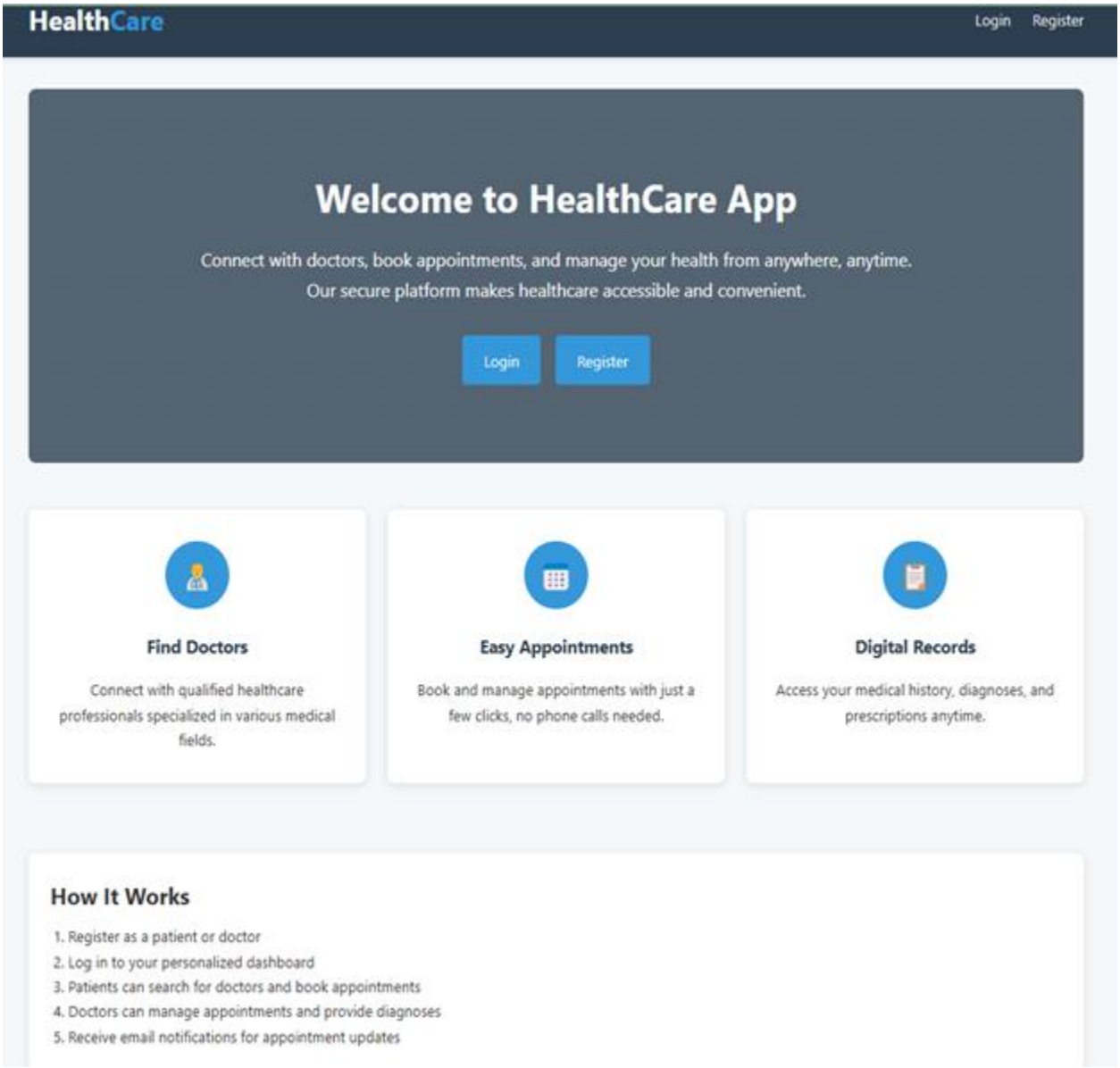
- Conduct functional testing to verify user registration, login, book requests, and notifications.

Home Page:

The Home Page of your project is the main entry point for users, where they can interact with the system. It typically includes:

- Input Fields: For users to enter basic information like appointment requests, diagnosis submissions, or service bookings
- Navigation: Links to other sections such as the login page, dashboard, or service options.
- Responsive Design: Ensures the page is accessible across devices with a clean, user-friendly interface.

The Home Page serves as the initial interface that directs users to the key functionalities of your web application.

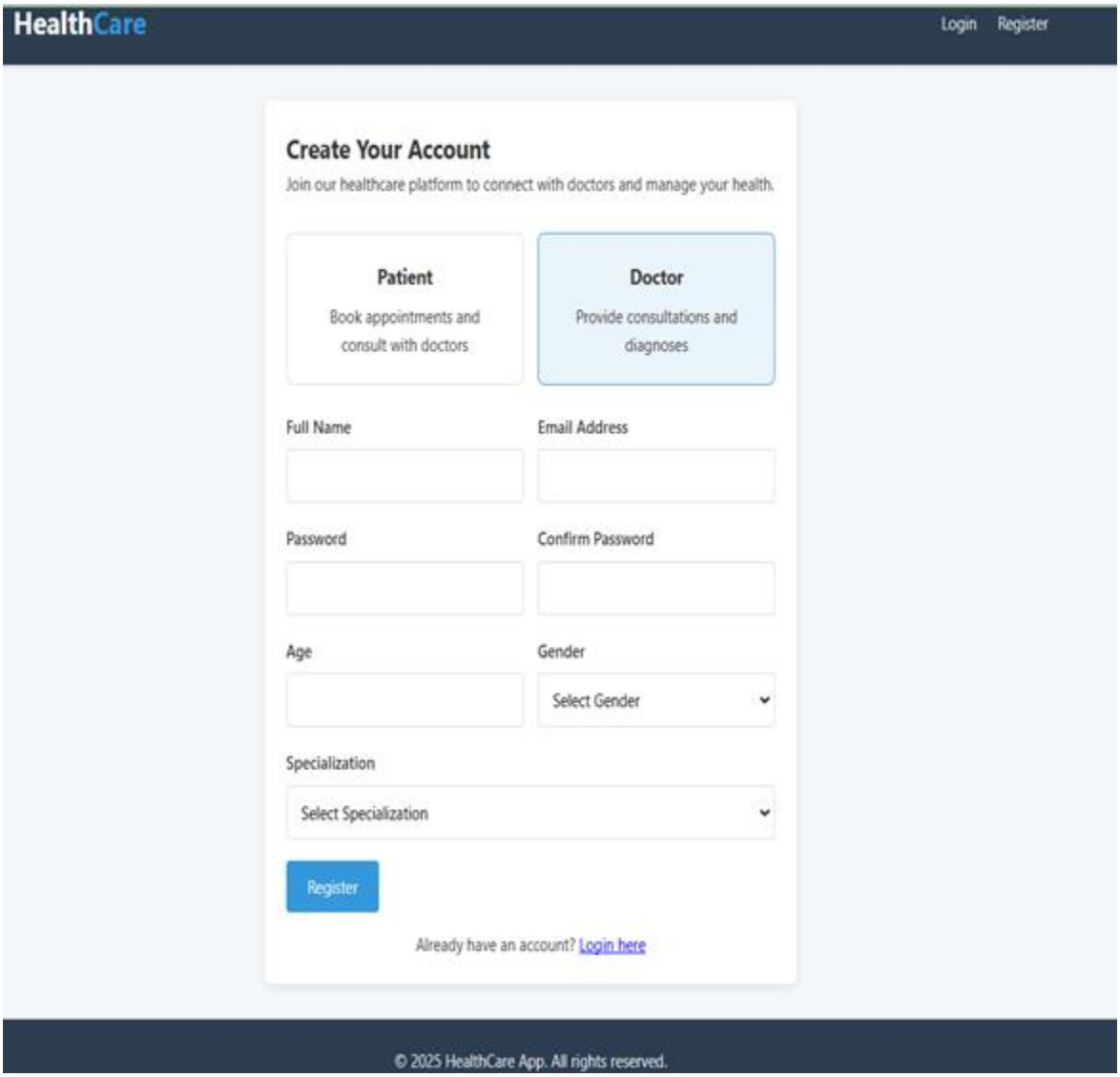


DOCTOR AND PATIENT REGISTRATION PAGE :

The Doctor Registration Page allows doctors to register and create an account on the platform. It typically includes:

- Input Fields: For doctor details such as name, specialty, qualifications, and contact information.
- Login Credentials: Fields for setting a username and password for secure access.

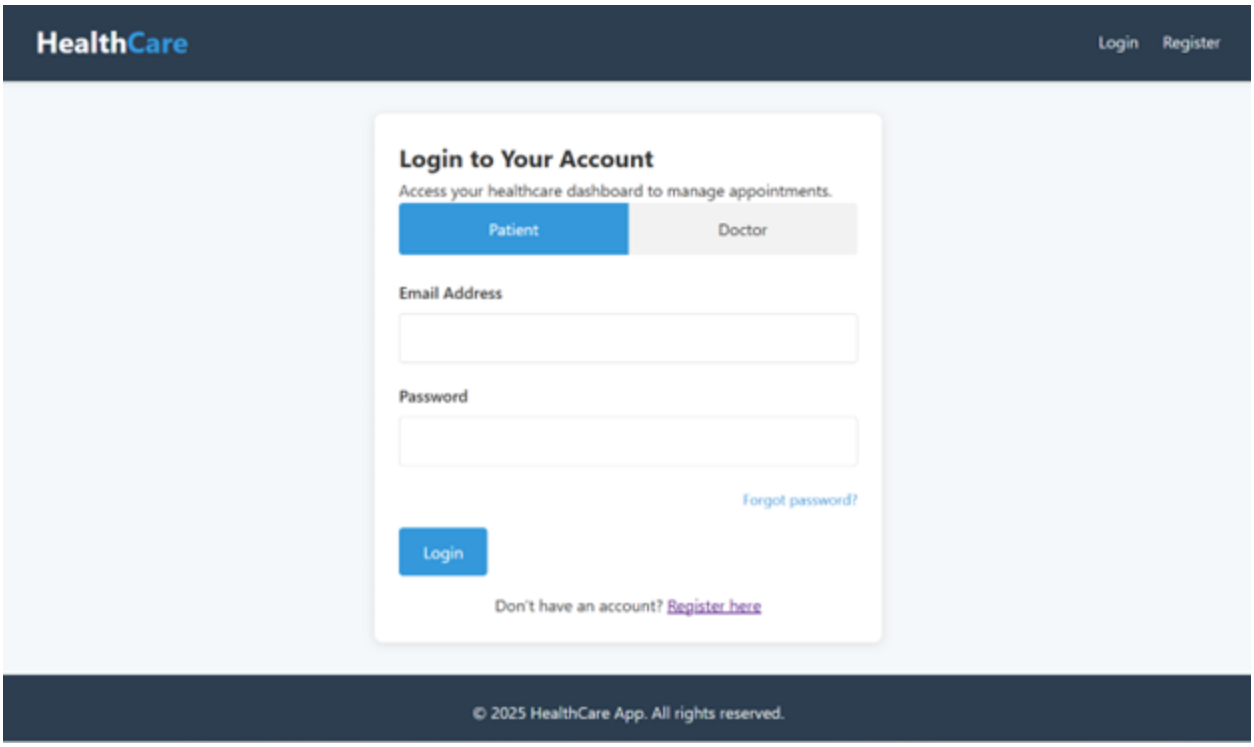
- Submit Button: A button to submit the registration details, which will then be stored in the database after validation.

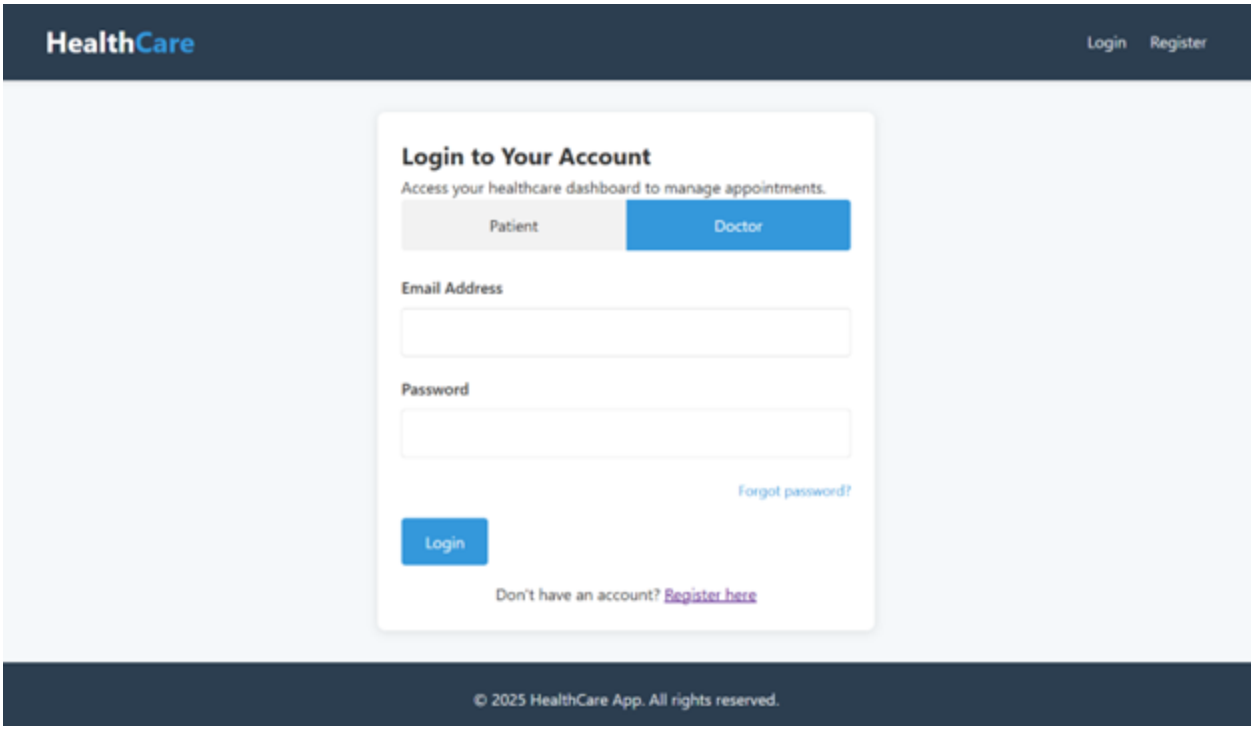


PATIENT AND DOCTOR LOGIN PAGES:

The Patient and Doctor Login Pages allow users to securely access their accounts on the platform. Each login page typically includes:

- Username and Password Fields: Users enter their credentials (username and password) to authenticate their account.
- Login Button: A button to submit login details and validate user access.
- Once logged in, patients and doctors are redirected to their respective dashboards to manage appointments, medical records, and other relevant tasks.





User Dashboard:

The User Dashboard (for patients) provides an easy interface to manage appointments and track their status. It typically includes:

- Book Appointment Section: A form for selecting a doctor, choosing an appointment time, and submitting the request.
- Appointment Status: A section showing the current status of appointments (e.g., confirmed, pending, or completed) with options to view details or cancel.
- Upcoming Appointments: A list of future appointments with relevant details such as doctor name, date, and time.

This dashboard helps patients book new appointments and keep track of their healthcare schedules.

SMARTBRIDGE
Let's Bridge the Gap

Smart Internz

HealthCare

Dashboard Profile Logout

Login successful.

Welcome, Kambham Prasanna Sai Ravi Teja

Manage your appointments and health records from your dashboard.

My Dashboard

Book New Appointment

1	0	1
Pending Appointments	Completed Appointments	Total Appointments

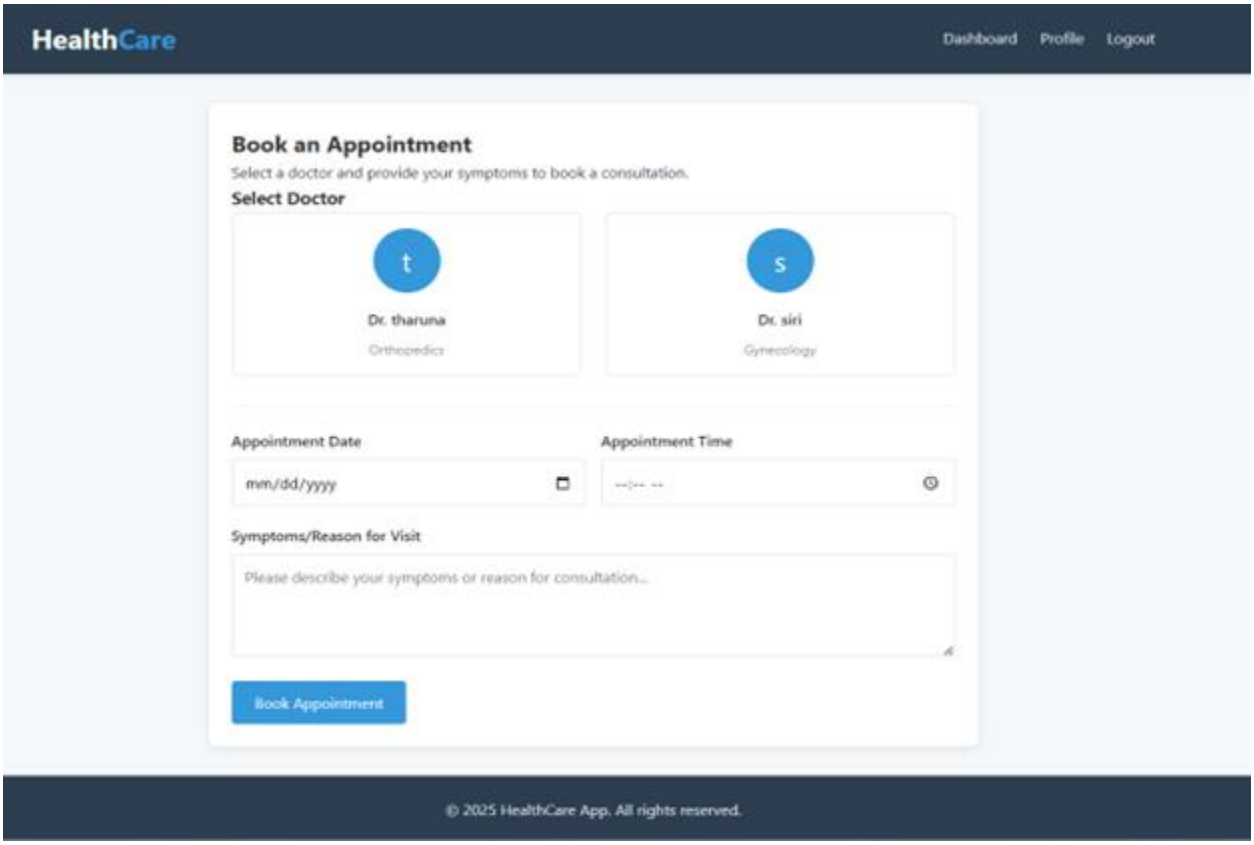
My Appointments | Available Doctors

My Appointments

Search doctor name or status... Search

Doctor	Date	Symptoms	Status	Actions
Dr. tharuna	2025-04-30	i am having issues with many things	Pending	View Details

© 2025 HealthCare App. All rights reserved.



Doctor Dashboard:

The Doctor Dashboard provides doctors with a comprehensive view of their upcoming appointments and patient details. It typically includes:

- Upcoming Appointments List: A table or list showing patient names, appointment times, and appointment statuses (e.g., confirmed, pending).
- Patient Details: Quick access to each patient's medical history, contact information, and previous visit records.
- Appointment Actions: Options to view, confirm, reschedule, or cancel appointments, ensuring efficient management.

The dashboard serves as the main interface for doctors to manage their schedules, track patient interactions, and provide timely care.

SMARTBRIDGE
Let's Bridge the Gap

Smart Internz

HealthCare

Dashboard Profile Logout

Login successful.

Welcome, Dr. tharuna

Manage your appointments and patient consultations from your dashboard.

Doctor Dashboard

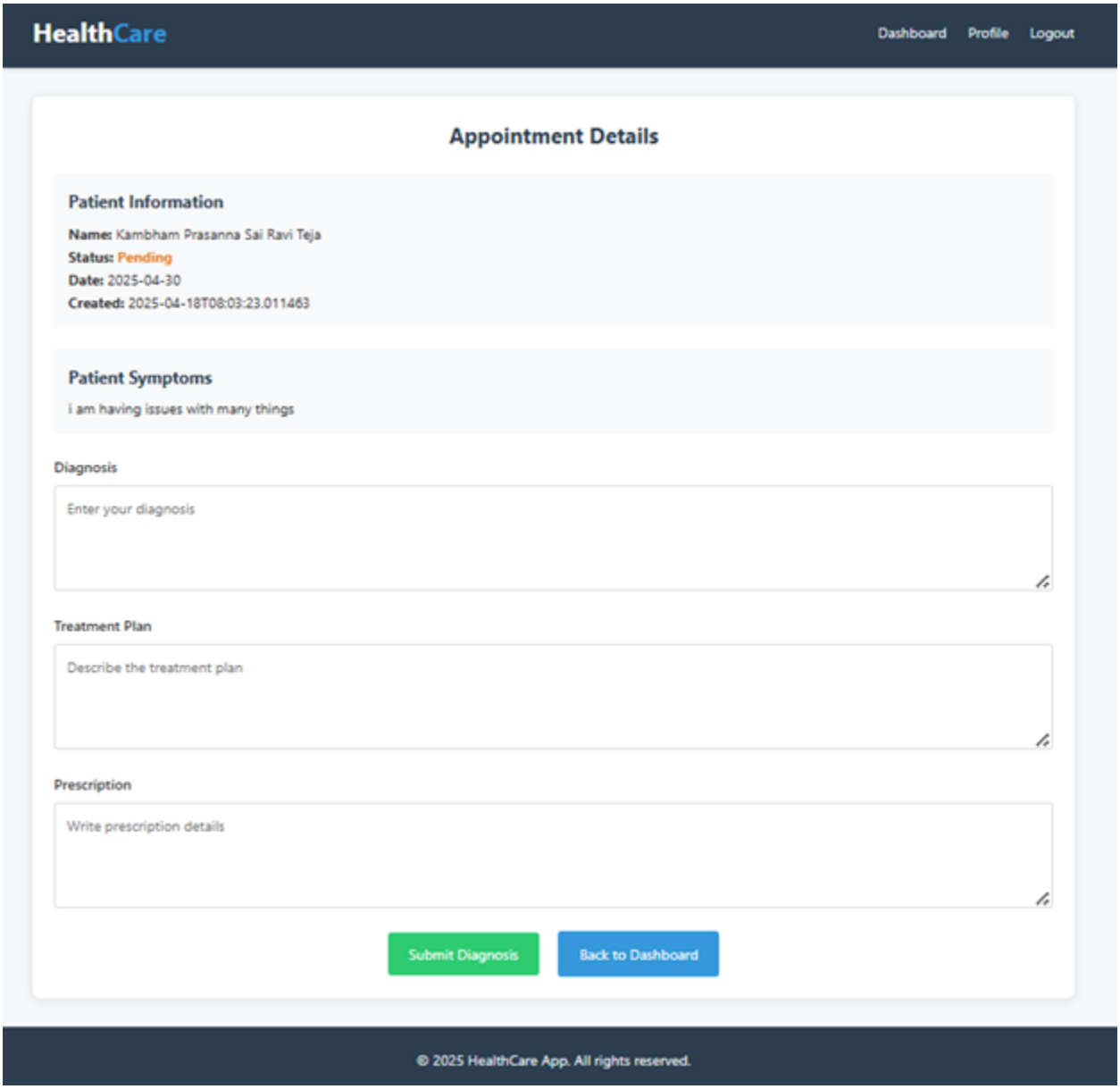
Search patient name... Search

1	0	1
Pending Appointments	Completed Appointments	Total Appointments

Pending Appointments | Completed Appointments | All Appointments

Patient Name	Date	Symptoms	Status	Actions
Kambham Prasanna Sai Ravi Teja	2025-04-30	i am having issues with many things	Pending	View Details

© 2025 HealthCare App. All rights reserved.



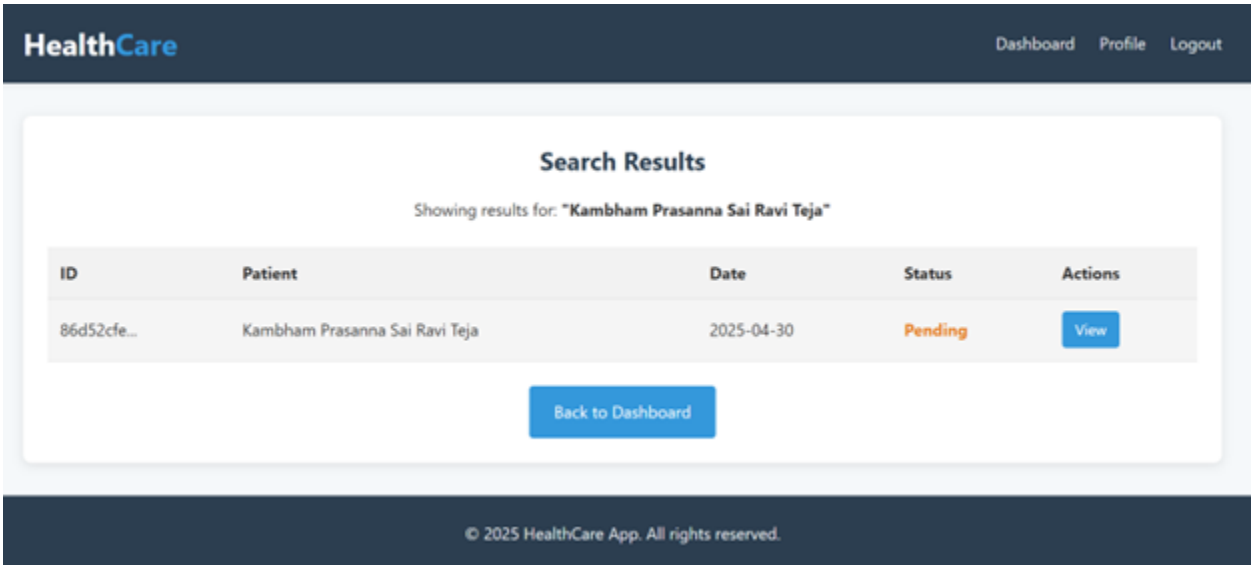
Search Appointment:

The Search Appointment feature in the Doctor Dashboard allows doctors to quickly find and manage specific patient appointments. It typically includes:

- Search Bar: A field where doctors can enter a patient's name, appointment date, or other relevant details to filter appointments.

- Appointment List: Displays the search results, including patient names, appointment times, and statuses, for easy access.
- Action Options: Options to view, update, or cancel appointments directly from the search results.

This feature streamlines appointment management, making it easier for doctors to find and handle patient requests.



Dynamodb Database updations :

Users table :

In the Users Table of DynamoDB, the data structure is designed to store user-related information for both patients and doctors. Typical updates include:

- Add New Users: When a new patient or doctor registers, their details such as name, email, role (patient/doctor), contact info, and password hash are added to the table.
- Update User Info: If a user updates their profile (e.g., changing contact details), the corresponding record in the table is modified.
- Status Tracking: Track the status of user accounts (active, inactive) based on their activity or admin updates.

The Users Table serves as the central repository for all user data, enabling quick access and modification of details when necessary.

Table: UsersTable - Items returned (4)

Scan started on April 18, 2025, 19:27:34

Actions ▾ | Create item

1

email *(String)*	age	created_at	gender	login_count	name
sunder@thesmartbrid...	25	2025-04-18...	male	2	sunder
gtharunasri19@gmail...	21	2025-04-18...	female	1	tharuna
sairaviteja478@gmail....	21	2025-04-18...	male	2	Kambhar
siri@thesmartbridge.c...	24	2025-04-18...	female	1	siri

Appointment table :

In the Appointment Table of DynamoDB, the data structure stores information related to patient appointments. Typical updates include:

- Add New Appointment: When a patient books an appointment, details such as patient ID, doctor ID, appointment date, time, and status (pending, confirmed, canceled) are stored.
- Update Appointment Status: As appointments are confirmed, rescheduled, or canceled, the status field in the table is updated accordingly.
- Appointment History: Historical data about completed appointments can also be stored to track past interactions between patients and doctors.

The Appointment Table allows for efficient management of appointments, ensuring accurate and up-to-date scheduling information for both doctors and patients.

Table: AppointmentsTable - Items returned (2)

Scan started on April 18, 2025, 19:27:09

Actions ▼ Create item

1

appointment_id *(String)*	appointment_date	created_at	diagnosis	doctor_email
86d52cfe-cce6-46b0-941d-1...	2025-04-30	2025-04-18...		gtharunasri19...
42e13fef-fbe9-42d1-9905-f...	2025-04-21	2025-04-18...	make sure t...	siri@thesmart...

- **Mail to the User:**

Welcome to HealthCare App Inbox ×

R **sairaviteja478@gmail.com**
to me

Welcome to HealthCare App, Kambham Prasanna Sai Ravi Teja! Your account has been created successfully.

- Mail to the admin:

Appointment Confirmation Inbox ×

R **sairaviteja478@gmail.com**
to me

Dear Kambham Prasanna Sai Ravi Teja,

Your appointment with Dr. tharuna has been booked successfully.

Date: 2025-04-30

Thank you for using our healthcare service.

Conclusion:

This section provides a concise summary of the AWS project's accomplishments and key outcomes. It reaffirms how the project addressed its initial objectives by leveraging specific AWS services and architecture. The conclusion highlights the benefits realized, such as improved scalability, cost efficiency, reliability, or enhanced security, and may also touch upon any challenges overcome. Finally, it offers insights into future opportunities or recommendations for further optimization and growth within the AWS environment.